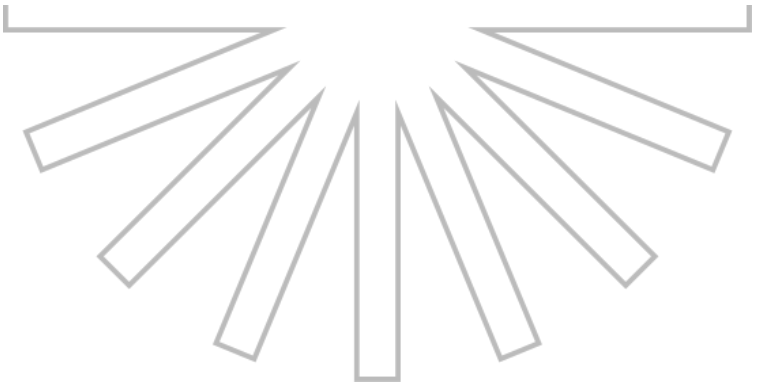
Deuda técnica en el desarrollo de software asistido por LLMs: un estudio de mapeo sistemático



Presentado a:

Robert Edison Arisaca Mamani

Presentado por:

- Carrasco Choque Arles Melvin
- Caceres Pari Angel Raul
- Chancuaña Alvis Klismann
- Yana Agramonte Luis Efrain



Contenido del documento

1. Introducción 3
2. Trabajos Relacionados 4
3. Planificación de la revisión 6
    3.1. Definir el tema de investigación 6
    3.2. Definir las preguntas de investigación (3 RQs) y palabras claves (Keywords) 6
    3.3. Estrategia PICOC 7
    3.4. Fuentes de Datos 7
    3.5. Identificar los Criterios de Inclusión (CI) y los Criterios de Exclusión (CE) 8
    3.6. Evaluación de la calidad 8
4. Conducción de la revisión 9
    4.1. Elaborar la cadena de búsqueda en Scopus y WoS 9
    4.2. Ejecución de la búsqueda en Scopus y Web of Science 10
    4.3. Selección de estudios primarios en Scopus y WoS 10
    4.4. Filtros y aplicación de CI/CE 14
    4.5. Estudios Primarios Seleccionados 14
5. Resultados 15
    5.1. Extraer los datos de investigación 15
    5.2. Estadísticas de la revisión 17
    5.3. RQ1: ¿Cómo han cambiado el concepto, los mecanismos de acumulación y la gestión de la deuda técnica? 20
    5.4. RQ2: ¿Qué métricas y técnicas se emplean para medir y detectar la deuda técnica, y qué efectividad reportan? 23
    5.5. RQ3: ¿Qué tipos de deuda técnica se reportan y cuáles emergen específicamente del desarrollo asistido por LLMs? 26
    5.6. Discusión 28
6. Conclusiones 28
7. Referencias 29
8. Anexos 31
9. 31

1. Introducción

La Guía del Cuerpo de Conocimiento de la Ingeniería de Software (SWEBOK) organiza el conocimiento generalmente aceptado de la disciplina en áreas de conocimiento. En su versión 4.0a, el área *Software Maintenance* aborda, en el tópico *Technical Debt Cost Estimation*, la deuda técnica: la describe como el esfuerzo necesario para corregir los problemas que permanecen en el código cuando una aplicación se libera, señala que encarece el mantenimiento más de lo necesario y propone estimarla a partir del tamaño, la complejidad y el número de fallos y violaciones de buenas prácticas de diseño y de codificación [1]. La guía añade que la deuda técnica no debe tratarse de forma aislada, sino examinando sus causas raíz.

El concepto tiene más de tres décadas. Cunningham [2] lo introdujo como una metáfora financiera: entregar código imperfecto para avanzar rápido equivale a contraer una deuda cuyo interés se paga como esfuerzo adicional en cada cambio posterior. La comunidad lo formalizó en el seminario Dagstuhl 16162 [3], y Potdar y Shihab [4] mostraron que los propios desarrolladores declaran parte de esa deuda en comentarios como TODO o FIXME, lo que dio origen a la línea de investigación sobre deuda técnica autoadmitida (SATD). La Sección 2 desarrolla estos conceptos.

En los últimos cinco años el contexto cambió de forma abrupta. La vista previa de GitHub Copilot (2021) y la publicación de ChatGPT (noviembre de 2022) llevaron los modelos de lenguaje grandes (LLMs) al trabajo diario de los desarrolladores, y desde 2025 los agentes autónomos como OpenAI Codex, Claude Code, Devin o Cursor Agent generan y envían pull requests completos con mínima intervención humana [5]. Según datos de GitHub citados por Patel et al. [6], Copilot llegó a producir más del 46 % del código en los archivos donde estaba habilitado. Este tema pertenece, por tanto, a la intersección más reciente entre la inteligencia artificial y la ingeniería de software, con apenas cuatro o cinco años de desarrollo: todos los registros recuperados en esta revisión se publicaron entre 2024 y 2026.

La aceleración tiene un reverso. Los estudios que reportaron ganancias de productividad con asistentes de código no evaluaron su efecto sobre la calidad, lo que lleva a preguntarse si se está desplegando a escala un nuevo tipo de generador de deuda técnica [7]. Si el código se produce más rápido de lo que los equipos pueden revisarlo y comprenderlo, cabe preguntar si la deuda resultante es la misma de siempre producida a mayor velocidad o si cambia su naturaleza; si los indicadores que propone el SWEBOK siguen capturándola; y si aparecen tipos de deuda que no existían antes de los LLMs.

Para responder esas preguntas, este trabajo realiza un estudio de mapeo sistemático ligero [8] siguiendo el procedimiento de Kitchenham y Charters [9] planificación, conducción y reporte sobre las bases de datos Scopus y Web of Science. El objetivo es caracterizar el estado del arte de la deuda técnica en el desarrollo de software asistido por LLMs: cómo evoluciona su concepto y su gestión, con qué métricas y técnicas se mide y detecta, y qué tipos de deuda se reportan.

El documento se organiza así: la Sección 2 presenta el marco teórico; la Sección 3, los trabajos relacionados; la Sección 4, la planificación de la revisión; la Sección 5, su conducción, con el detalle del proceso de búsqueda y selección; la Sección 6, los resultados, la caracterización de

los estudios, las respuestas a las preguntas de investigación y la discusión; la Sección 7, las conclusiones. Los anexos contienen el inventario de estudios primarios, el registro completo de decisiones y la plantilla de extracción.

2. Marco Teórico

2.1. Deuda técnica

La deuda técnica es el costo futuro que asume un equipo cuando elige una solución rápida o incompleta en lugar de una más adecuada. Cunningham [2] la describió con una analogía financiera: como un préstamo, permite avanzar más rápido hoy, pero genera intereses que se pagan como esfuerzo extra cada vez que hay que modificar ese código; si la deuda no se paga, los intereses se acumulan hasta dificultar cualquier cambio. De esa analogía provienen dos términos que se usan en toda la literatura: el principal, que es el esfuerzo necesario para corregir el problema, y el interés, que es el esfuerzo adicional que se paga mientras el problema permanece.

El seminario Dagstuhl 16162 consensuó la definición más citada: en sistemas intensivos en software, la deuda técnica es el conjunto de construcciones de diseño o implementación convenientes a corto plazo que crean un contexto técnico que puede hacer que los cambios futuros sean más costosos o imposibles; su impacto se limita a las cualidades internas del sistema, principalmente la mantenibilidad y la evolucionabilidad [3]. En otras palabras, la deuda técnica no es un error visible para el usuario el sistema funciona, sino un problema de estructura que encarece su mantenimiento.

El SWEBOK v4.0a la ubica dentro del área de mantenimiento de software y la vincula a la estimación de costos: la deuda representa el esfuerzo requerido para corregir los problemas que permanecen en el código al liberarlo, y puede medirse mediante indicadores de tamaño, complejidad y violaciones de buenas prácticas de diseño y codificación. Por eso, el SWEBOK recomienda que las organizaciones conozcan la calidad de su software, la magnitud de su deuda, el ahorro potencial de invertir en calidad y el impacto de los problemas actuales en el negocio [1]. Esta revisión se enmarca en ese tópico.

2.2. Tipos de deuda técnica

Kruchten, Nord y Ozkaya [10] mostraron que la deuda técnica no se limita al código: puede ser visible, como los defectos o las funcionalidades pendientes, o invisible, como las decisiones de diseño y arquitectura que solo notan los desarrolladores. A partir de esa idea, la literatura distingue varios tipos según el artefacto donde reside la deuda. Los más frecuentes son:

- Deuda de código: código duplicado, excesivamente complejo o que viola buenas prácticas (*code smells*).
- Deuda de diseño y de arquitectura: decisiones estructurales subóptimas, como dependencias cíclicas o módulos demasiado acoplados.
- Deuda de requisitos: requisitos incompletos, ambiguos o implementados solo parcialmente.

- Deuda de pruebas: cobertura insuficiente o pruebas frágiles.
- Deuda de documentación: documentación ausente, desactualizada o incorrecta.
- Deuda de defectos, de construcción (build) y de dependencias: errores conocidos que se posponen, procesos de compilación frágiles y dependencias inmaduras o desactualizadas.

2.3. Deuda técnica autoadmitida (SATD)

Una parte de la deuda la declaran los propios desarrolladores. Potdar y Shihab [4] llamaron deuda técnica autoadmitida (*Self-Admitted Technical Debt*, SATD) a los comentarios en los que el programador reconoce que dejó algo pendiente o mal resuelto, por ejemplo "TODO: optimizar esta consulta" o "FIXME: solución temporal". Como es texto escrito por personas, el SATD puede detectarse automáticamente con técnicas de procesamiento de lenguaje natural, lo que lo convirtió en una de las formas más estudiadas de deuda técnica. En el contexto de esta revisión, la pregunta es qué ocurre cuando quien escribe el código y el comentario es un LLM.

2.4. Desarrollo asistido por LLMs: chat, IDE, agentes y vibe coding

Un modelo de lenguaje grande (LLM) es un modelo de aprendizaje profundo entrenado con enormes cantidades de texto y código que, a partir de una instrucción en lenguaje natural (*prompt*), genera texto o código. No todos los estudios usan el LLM de la misma forma, y esa diferencia es importante para interpretar sus resultados. Agarwal et al. [5] distinguen dos paradigmas: los asistentes integrados en el IDE, que sugieren código en tiempo real dentro del editor y que el desarrollador acepta, modifica o rechaza mientras escribe; y los agentes autónomos, que operan a nivel de repositorio, planifican la tarea, modifican varios archivos y entregan su trabajo como un pull request completo con mínima intervención humana. A ellos se suma el uso más básico, el chat conversacional, en el que el desarrollador describe la tarea fuera del editor y copia la respuesta. La Tabla 1 resume las diferencias.

Modalidad	Cómo trabaja el desarrollador	Alcance del cambio	Ejemplos	Estudios del corpus
Chat conversacional	Describe la tarea en una ventana de chat, copia el código generado y lo adapta a mano.	Fragmentos o funciones	ChatGPT, Claude, Gemini, DeepSeek (chat)	S07
Asistente integrado en el IDE	Recibe sugerencias y autocompletado mientras escribe; decide en el momento qué aceptar. Trabajo síncrono.	Línea, función o archivo	GitHub Copilot, Cursor (modo editor), CodeWhisperer	S09; S01 (grupo de comparación)
Agente autónomo de codificación	Asigna una tarea completa; el agente planifica, edita varios archivos, ejecuta y abre un pull request. Trabajo asíncrono.	Repositorio completo	OpenAI Codex, Claude Code, Devin, Cursor Agent, SWE-agent	S01, S02, S03, S06

LLM como herramienta de gestión de deuda	No produce el software: se usa para clasificar, explicar o refactorizar la deuda existente.	Comentarios, commits o funciones	ChatGPT, GLM, LLMs de refactorización	S04, S08, S10

Tabla 1 - Modalidades de uso de los LLMs en el desarrollo de software

El vibe coding es la forma más extrema de esta tendencia. El término, popularizado a inicios de 2025, describe un estilo de programación en el que el desarrollador da instrucciones de alto nivel en lenguaje natural y el LLM genera el código paso a paso, en un ciclo iterativo de "pedir y responder", sin que el programador escriba ni revise el código en detalle. Bunch y Taeb [11] lo contrastan con el desarrollo tradicional, donde el desarrollador escribe, depura y optimiza el código manualmente, y señalan que su rol pasa de "escritor" a "director"; advierten que la naturaleza conversacional de este flujo facilita que código inseguro o incorrecto entre al proyecto sin la inspección habitual, lo que puede derivar en exceso de confianza y deuda técnica. Ninguno de los diez estudios seleccionados evalúa el vibe coding de forma empírica, pero los agentes autónomos (S01, S02, S03, S06) representan su versión más automatizada, y la deuda de comprensión descrita por S05 es su consecuencia más directa.

2.5. Términos, métricas y herramientas de medición

Los estudios primarios emplean términos y herramientas técnicas que conviene precisar antes de presentar los resultados. La Tabla 2 los define en el sentido en que se usan en este informe. Entre las herramientas destacan SonarQube y CodeQL, dos analizadores estáticos con enfoques distintos: SonarQube aplica un conjunto de reglas predefinidas y estima la deuda técnica como tiempo de remediación, mientras que CodeQL, desarrollado por GitHub, convierte el código en una base de datos y ejecuta consultas escritas en el lenguaje QL para encontrar patrones problemáticos; cada hallazgo se clasifica por severidad (error, advertencia o recomendación). S06 usó CodeQL para contar los problemas de mantenibilidad y seguridad que introducían los parches generados por LLMs.

Término o herramienta	Significado en este informe	Estudios
Análisis estático	Examen automático del código sin ejecutarlo, para detectar defectos, vulnerabilidades o malas prácticas.	S01, S03, S06, S07, S08
SonarQube	Plataforma de análisis estático con reglas de mantenibilidad, fiabilidad y seguridad; expresa la deuda como tiempo de remediación (minutos, horas o días).	S01, S07
CodeQL	Motor de análisis de GitHub que trata el código como una base de datos consultable mediante el lenguaje QL; reporta hallazgos de seguridad y mantenibilidad por severidad (error, advertencia, recomendación).	S06
Radon	Herramienta para Python que calcula la complejidad ciclomática y métricas brutas como líneas de código.	S03
CodeHealth (CodeScene)	Métrica agregada que resume más de veinte indicadores de mantenibilidad, como líneas de código y complejidad ciclomática.	S08

Code smell	Síntoma en el código que indica un problema de diseño (método demasiado largo, lógica muy anidada, duplicación).	S02, S08
Complejidad ciclomática	Número de caminos de ejecución independientes de una función; a más caminos, más difícil de probar.	S03, S07, S08
Complejidad cognitiva	Medida de lo difícil que es entender el flujo de control; penaliza especialmente el anidamiento.	S01, S07
Clones de código (Tipo 1 a 4)	Código repetido. Tipo 1: copia exacta; Tipo 2: con nombres cambiados; Tipo 3: con modificaciones; Tipo 4: misma funcionalidad escrita de otra forma (clon semántico).	S03
Pull request (PR)	Propuesta de cambio que se envía a un repositorio para su revisión antes de integrarse.	S01, S02, S03, S04
SWE-bench Lite	Conjunto de 300 issues reales de 11 proyectos Python de GitHub usado para evaluar si un LLM resuelve problemas reales.	S06
AIDev	Conjunto de datos público de pull requests creados por agentes de IA en GitHub.	S01, S02, S03
Ajuste fino (fine-tuning)	Reentrenar un modelo existente con datos de la tarea específica.	S04
Aprendizaje en contexto (few-shot)	Usar el modelo sin reentrenarlo, dándole solo instrucciones y algunos ejemplos en el prompt.	S04
LLM como evaluador (LLM-as-a-judge)	Usar un LLM para puntuar o clasificar salidas en lugar de un evaluador humano.	S02
Precisión, exhaustividad y F1	Precisión: proporción de detecciones correctas. Exhaustividad (recall): proporción de casos reales que se detectan. F1: media armónica de ambas.	S04, S08
AUC	Área bajo la curva ROC; mide la capacidad de un clasificador para distinguir casos positivos de negativos.	S10
Kappa de Cohen	Medida del acuerdo entre dos evaluadores que descuenta el acuerdo por azar; > 0,8 se considera excelente.	S09
Diferencias en diferencias	Método estadístico que compara la evolución de un grupo que adopta una práctica con la de un grupo de control similar, para estimar su efecto causal.	S01
Mann-Whitney U / Kruskal-Wallis	Pruebas estadísticas no paramétricas para comparar dos o más grupos.	S02, S03

Tabla 2 - Términos, métricas y herramientas empleados por los estudios primarios

3. Trabajos Relacionados

La misma búsqueda que recuperó los estudios primarios identificó cuatro estudios secundarios cercanos al tema. Siguiendo la indicación de no confundir trabajos relacionados con estudios

que ya responden las preguntas planteadas, se verificó que ninguno de ellos aborda de forma central la deuda técnica que introduce el desarrollo asistido por LLMs ni la manera de medirla; por esa razón no invalidan esta revisión, sino que la enmarcan. A continuación se discuten los tres más relevantes.

Estudio	Tipo	Alcance	Enfoque	Diferencia con esta revisión
Moreschini et al. (2026)	Revisión multivocal	61 fuentes	Gestión de deuda técnica en SE4AI y AI4SE	Estudia cómo adaptar la gestión de deuda en sistemas de IA.
Gosu et al. (2026)	RSL	29 estudios	IA generativa como herramienta para gestionar deuda técnica	Se centra en el LLM como instrumento de gestión.
Abdulaziz et al. (2026)	RSL	46 estudios (de 1 537)	Detección automatizada de SATD, 2015–2025	Revisa técnicas de detección en software general.
Kosmas et al. (2026)	Revisión (PRISMA)	22 estudios (de 12 450)	Ciclo de vida de ML y MLOps	La deuda técnica es un tema secundario dentro del ciclo de vida de ML.

Tabla 3 - *Estudios secundarios identificados en la búsqueda*

Moreschini et al. [12]: la evolución de la deuda técnica de DevOps a la IA generativa
Es la revisión más cercana. Mediante una revisión multivocal de 61 fuentes académicas e industriales, los autores estudian cómo debe adaptarse la gestión de la deuda técnica en dos direcciones: la ingeniería de software para sistemas de IA (SE4AI) y el uso de IA dentro del proceso de ingeniería (AI4SE). Concluyen que la deuda relacionada con datos, infraestructura y pipelines es particularmente prevalente en los sistemas de aprendizaje automático, que las prácticas de MLOps son esenciales para gestionarla y que los artefactos generados por IA introducen nuevos desafíos de gobernanza y mantenibilidad. Su aporte es un mapa de la intersección entre IA y gestión de deuda. Sin embargo, su foco principal es la deuda de los sistemas de IA; no sintetiza evidencia primaria sobre cuánta deuda introduce el código generado por LLMs ni compara métricas o técnicas para detectarla, y combina literatura gris con literatura revisada por pares.

Gosu et al. [13]: la IA generativa en la gestión de la deuda técnica
Esta revisión sistemática analiza 29 estudios sobre el uso de IA generativa para gestionar deuda técnica. Encuentra que los enfoques actuales apoyan sobre todo la identificación y el pago de la deuda mediante modelos basados en prompts, y que las actividades de gestión más amplias, las comparaciones arquitectónicas y la colaboración humana siguen poco exploradas. Trata al LLM exclusivamente como instrumento de gestión. La presente revisión cubre además la otra cara: el LLM como fuente de deuda, que es el rol dominante en los estudios primarios seleccionados (7 de 10).

Abdulaziz et al. [14]: detección automatizada de SATD

Revisa 46 estudios primarios, seleccionados de 1 537 registros, sobre la detección automatizada de deuda técnica autoadmitida entre 2015 y 2025. Documenta la evolución desde modelos basados en palabras clave y características manuales hacia arquitecturas contextuales basadas en transformers, y advierte que su adopción está limitada por la escasez de benchmarks, los protocolos de evaluación inconsistentes y la falta de explicabilidad. Su alcance es la detección de SATD en software en general; no distingue el código producido con asistencia de LLMs ni evalúa si los detectores entrenados con comentarios humanos funcionan sobre deuda generada por agentes.

El cuarto estudio, Kosmas et al. [15], revisa el ciclo de vida del aprendizaje automático y trata la deuda técnica solo de forma tangencial. En conjunto, los trabajos relacionados dejan una brecha precisa: no existe una síntesis de estudios primarios revisados por pares sobre cómo cambia, cómo se mide y qué tipos adopta la deuda técnica cuando el código se desarrolla con asistencia de LLMs. Esa brecha es el objeto de esta revisión.

4. Planificación de la revisión

La planificación sigue las etapas obligatorias de Kitchenham y Charters [9]: especificar las preguntas de investigación y desarrollar el protocolo de revisión.

4.1. Definir el tema de investigación

Tema: Deuda técnica en el desarrollo de software asistido por modelos de lenguaje grandes (LLMs).

Área de conocimiento SWEBOK v4: *Software Maintenance*, *Technical Debt Cost Estimation*.

Objetivo general: caracterizar el estado del arte sobre la deuda técnica en el desarrollo de software asistido por LLMs, identificando cómo cambian su concepto y su gestión, qué métricas y técnicas se usan para medirla y detectarla, y qué tipos de deuda se reportan.

4.2. Definir las preguntas de investigación (3 RQs) y palabras claves (Keywords)

Las preguntas buscan entender el estado del arte, no fundamentos teóricos: no preguntan qué es la deuda técnica, sino qué se sabe y qué no de su comportamiento cuando interviene un LLM. Cada pregunta se formula en español y en inglés, como exige el protocolo.

ID	Pregunta de investigación	Motivación
RQ1	¿Cómo han cambiado el concepto, los mecanismos de acumulación y la gestión de la deuda técnica con la incorporación de los LLMs y la IA generativa al desarrollo de software? *How have the concept, the accumulation mechanisms and the management of technical debt changed with the incorporation of LLMs and generative AI into software development?*	Identificar si la deuda técnica asistida por LLMs es la misma deuda de siempre producida más rápido, o si cambia la forma en que se origina, se acumula y se gestiona. Es la base para decidir si los enfoques clásicos de gestión (SWEBOK, Cap. 7) siguen siendo suficientes.

RQ2	¿Qué métricas y técnicas se emplean para medir y detectar la deuda técnica en el desarrollo asistido por LLMs, y qué evidencia de efectividad reportan? *What metrics and techniques are used to measure and detect technical debt in LLM-assisted software development, and what evidence of effectiveness do they report?*	El SWEBOK (Cap. 7) indica que la deuda técnica se estima a partir de tamaño, complejidad y violaciones de buenas prácticas. Se busca saber si esas métricas y las técnicas de detección existentes capturan la deuda que introduce el código generado por LLMs y con qué fiabilidad.
RQ3	¿Qué tipos de deuda técnica se reportan en el desarrollo asistido por LLMs y cuáles emergen específicamente de este contexto frente a la deuda técnica clásica? *What types of technical debt are reported in LLM-assisted software development, and which ones emerge specifically from this context compared to classical technical debt?*	Clasificar la deuda que aparece al integrar código generado por LLMs y distinguir los tipos clásicos (código, diseño, requisitos, pruebas, documentación) de las categorías nuevas propuestas en la literatura reciente.

Tabla 4 - Preguntas de investigación y su motivación

4.3. Estrategia PICOC

Del análisis de las tres RQs emergen tres bloques conceptuales presentes en todas ellas, deuda técnica, LLMs o IA generativa y desarrollo de software, cada uno con sus sinónimos. Se obvia la comparación porque es un estudio de Mapeo.

En la siguiente imagen se muestra el proceso de picoc

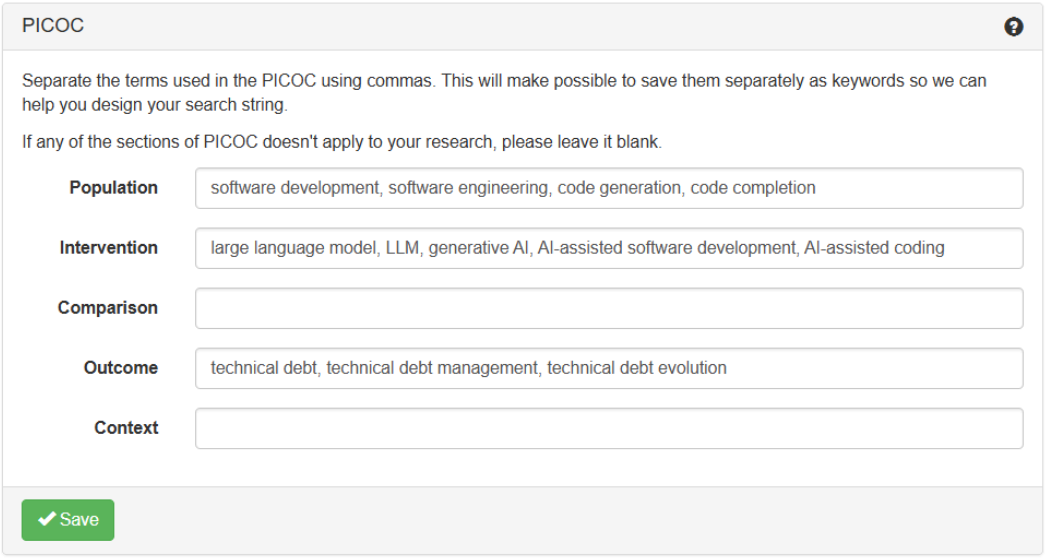


Figura 1 - Proceso PICOC

Elemento	Definición en esta revisión	Términos de búsqueda
Population (Población)	Proyectos y equipos de desarrollo de software (código abierto e industrial)	software development, software engineering, code generation, code completion

Intervention (Intervención)	Desarrollo asistido por LLMs, IA generativa, asistentes y agentes de codificación	large language model*, LLM*, generative AI, AI-assisted software development, AI-assisted coding
Outcome (Resultado)	Deuda técnica: concepto, métricas, técnicas de detección y tipologías	technical debt, technical debt management, technical debt evolution
Context (Contexto)	Mantenimiento y evolución del software (SWEBOK v4, Cap. 7)	No tiene

Tabla 5 - Estrategia PICOC de la revisión

Término clave	Términos relacionados
Deuda técnica	technical debt; technical debt management; technical debt evolution; self-admitted technical debt (SATD)
Modelos de lenguaje / IA generativa	large language model*; LLM*; generative AI; AI-assisted software development; AI-assisted coding
Desarrollo de software	software development; software engineering; code generation; code completion

Tabla 6 - *Términos clave y términos relacionados*

4.4. Fuentes de Datos

Se seleccionaron dos base de datos, en este caso Scopus y Web of Science

Fuente	URL	Justificación
Scopus	https://www.scopus.com	Base multidisciplinaria de mayor cobertura, indexa ACM, IEEE, Elsevier y Springer, e incluye preprints recientes.
Web of Science (Core Collection)	https://www.webofscience.com	Base de revistas y actas con control editorial estricto, complementa la cobertura de Scopus.

Tabla 7 - *Fuentes de Datos*

4.5. Identificar los Criterios de Inclusión (CI) y los Criterios de Exclusión (CE)

Los criterios se derivan de las preguntas de investigación, un registro se incluye si cumple todos los criterios de inclusión (Tabla 6) y ninguno de exclusión (Tabla 7). La ventana temporal parte de 2021 porque la vista previa de GitHub Copilot marca el inicio del desarrollo asistido por LLMs a escala.

ID	Descripción
CI1	Estudio primario: empírico, propuesta de técnica o herramienta evaluada, o contribución conceptual original.

CI2	Aborda la relación entre deuda técnica y LLMs/IA generativa: deuda introducida por el desarrollo asistido por LLMs, o LLMs empleados para detectar, medir o remediar deuda técnica.
CI3	Publicado entre 2021 y 2026 (desde la vista previa pública de GitHub Copilot, junio de 2021).
CI4	Artículo de revista o de conferencia con revisión por pares.
CI5	Escrito en inglés o español.
CI6	Aporta evidencia para al menos una pregunta de investigación.

Tabla 8 - Criterios de inclusión

ID	Descripción
CE1	Estudio secundario o terciario (RSL, revisión multivocal, mapeo, survey de literatura). Se traslada a Trabajos relacionados.
CE2	Registro duplicado (misma obra en ambas bases o en versiones preprint y publicada). Se conserva la versión publicada más completa (Kitchenham, §6.4.4).
CE3	Preprint, manuscrito sin revisión por pares, editorial, póster o resumen extendido.
CE4	La deuda técnica o los LLMs/IA generativa aparecen solo de forma tangencial (motivación o contexto) y no son objeto de estudio.
CE5	Fuera del dominio de la ingeniería de software.
CE6	Texto completo no disponible para el equipo.

Tabla 9 - Criterios de exclusión

4.6. Evaluación de la calidad

recomiendan evaluar la calidad de los estudios primarios para obtener criterios de inclusión más detallados, las preguntas más apropiadas para cada revisión, se construyó un instrumento de seis preguntas que cubre las etapas de diseño, conducción, análisis y conclusiones de un estudio, más una pregunta de relevancia.

ID	Pregunta
QA1	¿Los objetivos o preguntas de investigación están claramente definidos?
QA2	¿El diseño o método de investigación está descrito y es adecuado para el objetivo?
QA3	¿Los datos o la muestra están descritos y son suficientes (origen, tamaño, representatividad)?
QA4	¿Los resultados se sustentan en un análisis riguroso (pruebas estadísticas, métricas de evaluación o validación)?
QA5	¿Se discuten las amenazas a la validez o las limitaciones del estudio?
QA6	¿El estudio aporta evidencia a las RQs de esta revisión?

Tabla 10 - Criterios de exclusión

5. Conducción de la revisión

5.1. Elaborar la cadena de búsqueda en Scopus y WoS

La cadena se construyó en cuatro pasos. Primero, se extrajeron de las tres RQs los conceptos que todas comparten. Segundo, se reunieron sinónimos y variantes de cada concepto, usando truncamiento para capturar plurales y derivados (`large language model` *recupera* modelos *y* modeling*;* `*LLM*` recupera *LLMs*). Tercero, los términos de un mismo bloque se unieron con OR y los bloques entre sí con AND, de modo que un registro debe tratar a la vez de IA generativa, de desarrollo de software y de deuda técnica.

Cuarto, la búsqueda se restringió a título, resumen y palabras clave (campo TITLE-ABS-KEY en Scopus y TS en WoS) para evitar registros que solo mencionan los términos de paso en el texto completo.

La cadena se refinó en tres iteraciones. La primera, con solo dos bloques, recuperaba estudios de deuda técnica en sistemas de aprendizaje automático y usos de LLMs ajenos a la construcción de software. La segunda añadió el bloque de desarrollo de software, pero perdía los estudios que hablan de *code generation* o de *AI-assisted coding* sin usar la palabra *LLM*.

La tercera incorporó esos sinónimos y las variantes explícitas de deuda que aparecen en la formulación de la RQ1 (*technical debt management*, *technical debt evolution*), se mantienen por explicitud aunque la frase «technical debt» ya las recupera.

Como resultado del proceso iterativo de refinamiento de la estrategia de búsqueda, se definió la cadena final utilizada para la recuperación de estudios primarios. La Figura X presenta una captura de pantalla de la cadena de búsqueda definitiva empleada en las bases de datos seleccionadas.

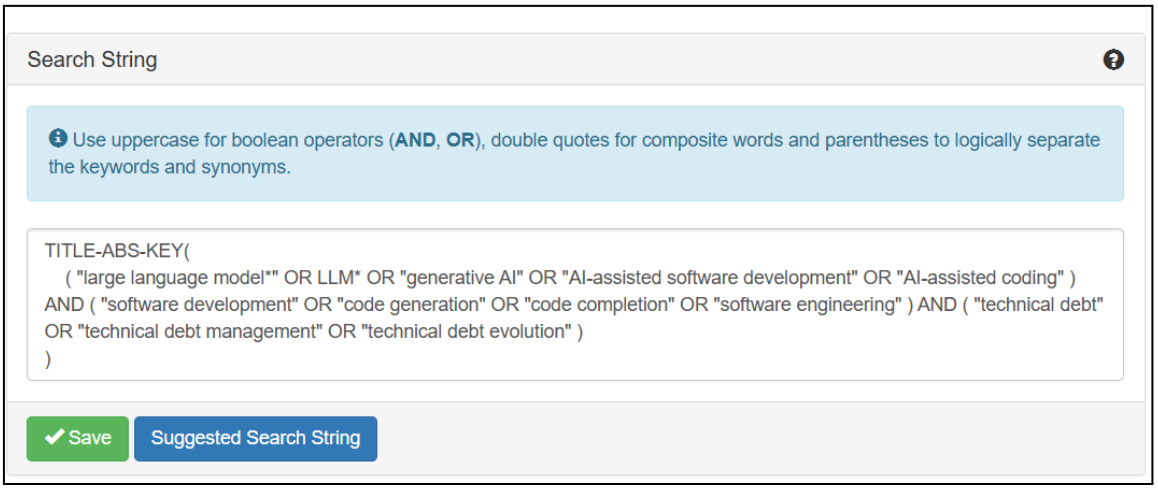


Figura 2 - Muestra la cadena final de búsqueda

Iter.	Cambio respecto a la anterior	Cadena (TITLE-ABS-KEY / TS)	Problema detectado
I1	Cadena inicial con dos bloques: IA (LLM, generative AI) AND deuda técnica.	("large language model*" OR LLM* OR "generative AI") AND ("technical debt")	Recupera estudios de deuda técnica en sistemas de ML y usos de LLM ajenos a la construcción de software.

I2	Se añade el bloque de contexto de ingeniería de software para acotar al desarrollo de software.	("large language model*" *OR LLM* OR "generative AI") AND ("software development" OR "software engineering") AND ("technical debt")	Pierde estudios que hablan de generación o completado de código sin usar la palabra 'software development', y los que hablan de 'AI-assisted coding' sin mencionar LLM.
I3	Se añaden sinónimos de la intervención (AI-assisted software development, AI-assisted coding) y del contexto (code generation, code completion), y variantes explícitas de deuda (management, evolution) derivadas de las RQs.	TITLE-ABS-KEY (("large language model*" OR LLM* OR "generative AI" OR "AI-assisted software development" OR "AI-assisted coding") AND ("software development" OR "code generation" OR "code completion" OR "software engineering") AND ("technical debt" OR "technical debt management" OR "technical debt evolution"))	

Tabla 11 - Iteraciones de la cadena de búsqueda

5.2. Ejecución de la búsqueda en Scopus y Web of Science

La cadena final adaptada a la sintaxis de cada base. La búsqueda se ejecutó sin filtros de tipo documental ni de año para no perder estudios muy recientes, Scopus devolvió 16 registros y WoS 18, para un total de 34 registros identificados.

- Búsqueda en WoS (Web of Science)

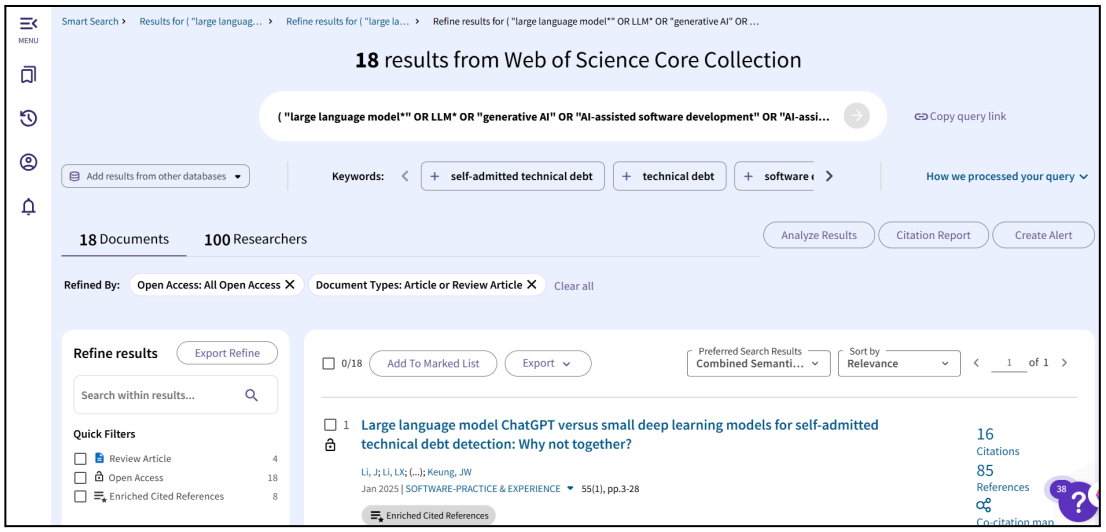


Figura 3 - Resultados Web of Science

Se realizó la búsqueda de documentos en Web of Science (WoS); posteriormente, tras aplicar los filtros correspondientes, se obtuvo un total de 18 documentos primarios.

- Búsqueda en Scopus

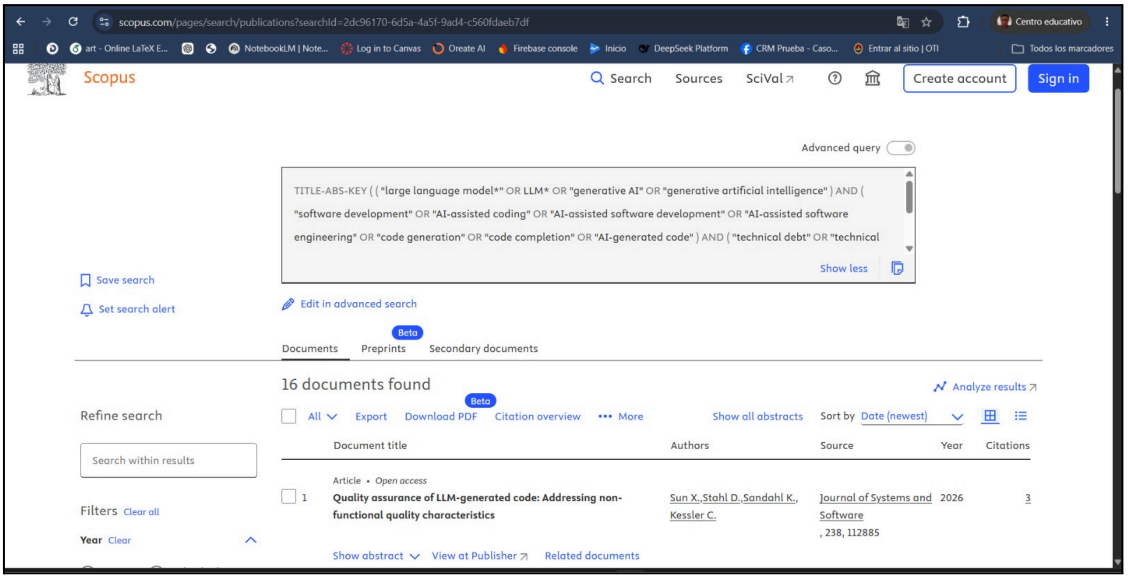


Figura 4 - Resultados Scopus

Se realizó la búsqueda bibliográfica en la base de datos Scopus. Tras la aplicación de los criterios de inclusión y los filtros correspondientes, se identificó un total de 16 documentos primarios para el análisis.

Tras consolidar los documentos obtenidos de las bases de datos Web of Science (WoS) y Scopus, se obtuvo un total de 34 documentos primarios. La distribución de estos resultados puede observarse en la siguiente Tabla 10.

Base	Cadena de búsqueda final	Registros	Fecha
Scopus	TITLE-ABS-KEY (("large language model*" *OR LLM* OR "generative AI" OR "AI-assisted software development" OR "AI-assisted coding") AND ("software development" OR "code generation" OR "code completion" OR "software engineering") AND ("technical debt" OR "technical debt management" OR "technical debt evolution"))	16	sept. 2026
Web of Science	TS=(("large language model*" *OR LLM* OR "generative AI" OR "AI-assisted software development" OR "AI-assisted coding") AND ("software development" OR "code generation" OR "code completion" OR "software engineering") AND ("technical debt" OR "technical debt management" OR "technical debt evolution"))	18	sept. 2026
Total		34	

Tabla 10 - Se muestra la cantidad de documentos de la búsqueda en WoS y Scopus.

5.3. Selección de estudios primarios en Scopus y WoS

La selección se realizó en varias etapas, una primera lectura liberal ayudada con IA, para la revisión sobre los requisitos de calidad, seguida de la lectura del texto completo y de una etapa final basada en criterios de calidad. La Figura 1 resume el flujo y la Tabla 11 muestra cuántos registros permanecen en cada etapa según su base de origen.

Etapa	Solo Scopus	Scopus y WoS	Solo WoS	Total únicos	Presentes en Scopus	Presentes en WoS
1.Registros identificados (exportados)				34	16	18
1. Tras eliminar duplicados	8	8	10	26	16	18
2. Cribado por título, resumen y conclusiones	7	6	3	16	13	9
3. Elegibles tras lectura del texto completo	7	4	3	14	11	7
4. Evaluados en calidad	7	4	1	12	11	5
Estudios primarios seleccionados	5	4	1	10	9	5

Tabla 11 - Número de registros por etapa de selección y base de origen

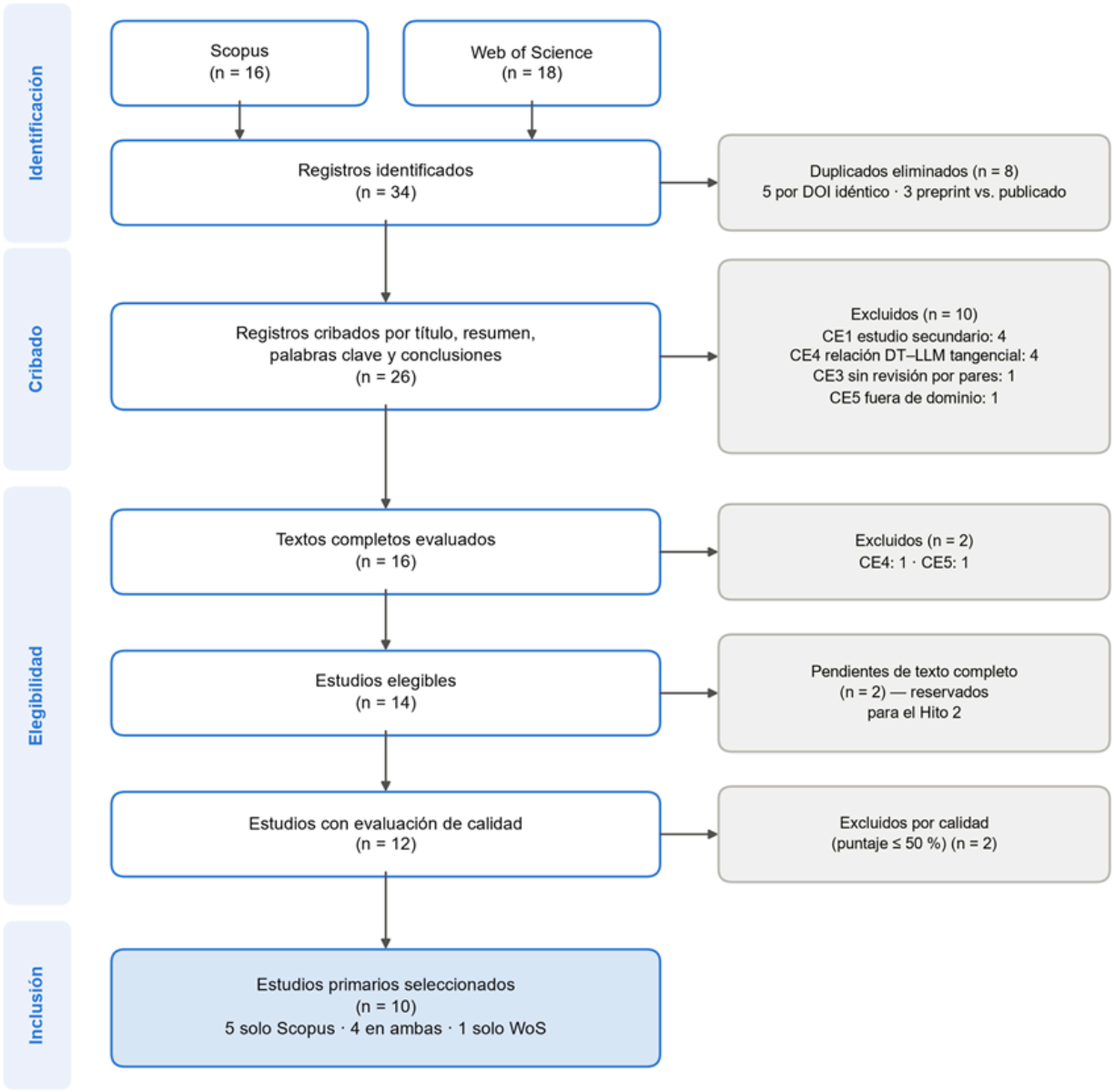


Figura 1 - Proceso de selección de estudios primarios

Etapa 1: identificación y eliminación de duplicados

Los 34 registros se compararon por DOI y por título. Se encontraron 8 duplicados. Cinco tenían DOI idéntico en ambas bases. Los otros tres eran más difíciles de detectar, porque no comparten DOI: Scopus los había indexado como preprint o manuscrito y WoS como artículo ya publicado en revista. Se conservó en todos los casos la versión publicada más completa. Quedaron 26 registros únicos: 8 solo de Scopus, 10 solo de WoS y 8 presentes en ambas bases.

Registro	Estudio	IDs en las bases	Tipo de duplicado	Resolución
R01	Abdulaziz et al. Automated detection of SATD: A SLR	SC01 / WS10	DOI idéntico	Se conserva un registro
R15	MacArthur LLMs and the problem of rhetorical debt	SC07 / WS05	DOI idéntico	Se conserva un registro

R14	Li, Q. et al. IMPACT	SC06 / WS15	DOI idéntico	Se conserva un registro
R18	Patel et al. Code comprehension gaps	SC10 / WS14	DOI idéntico	Se conserva un registro
R26	Wang et al. Just-in-time TODO-missed commits detection	SC16 / WS18	DOI idéntico	Se conserva un registro
R16	Moreschini et al. Evolution of TD from DevOps to GenAI	SC08 / WS03	Preprint vs. versión publicada	Se conserva JSS 231 (WoS)
R21	Simon et al. Automated detection of algorithm debt	SC11 / WS09	Preprint (arXiv 2024) vs. versión publicada	Se conserva EMSE 31(3) (WoS)
R23	Sun et al. Quality assurance of LLM-generated code	SC13 / WS06	Preprint (arXiv) vs. versión publicada	Se conserva JSS 238 (WoS)

Etapa 2: cribado por título, resumen, palabras clave y conclusiones

Cada registro se evaluó con los criterios CI/CE a partir de su título, resumen y palabras clave. Siguiendo la advertencia de Brereton et al., citada por Kitchenham y Charters [9], sobre la baja calidad de los resúmenes en ingeniería de software, también se leyeron las conclusiones. Se aplicó un criterio liberal: ante la duda, el registro pasaba a la lectura completa. Se excluyeron 10 registros: cuatro estudios secundarios (CE1), que se trasladaron a Trabajos relacionados; cuatro en los que la deuda técnica o los LLMs eran solo contexto (CE4); un manuscrito sin revisión por pares (CE3) y uno fuera del dominio de la ingeniería de software (CE5).

Registro	Estudio	Criterio	Motivo de exclusión
R01	Abdulaziz et al. (2026). Automated detection of self-admitted technical debt: A systematic literature review	CE1	Revisión sistemática (46 estudios primarios de 1 537 registros, 2015–2025). Se traslada a Trabajos relacionados.
R03	Bagane et al. (2025). Automatic detection of technical debt in large-scale Java codebases: A multi-model deep learning methodology for enhanced software quality	CE4	Detecta deuda técnica con RF, GBM, LSTM y GRU; no interviene ningún LLM ni IA generativa.
R04	Beaucoral (2026). Cracking the code: Enhancing development finance understanding with artificial intelligence	CE5	Aplica BERTopic a narrativas de proyectos de cooperación (OCDE); 'code' alude a códigos de proyectos, no a software.
R05	Bhatnagar (2026). Modernization of enterprise payment infrastructure: A case study on LLM-assisted migration of legacy distributed systems	CE4	La deuda técnica es solo motivación; los resultados miden tiempo de entrega, latencia y eventos anómalos de la migración.

R10	Gosu et al. (2026). The use of generative AI in technical debt management: A systematic literature review	CE1	Revisión sistemática (29 estudios) sobre IA generativa en la gestión de deuda técnica. Se traslada a Trabajos relacionados.
R12	Kosmas et al. (2026). Machine learning lifecycle: A survey	CE1	Revisión PRISMA (22 estudios) sobre el ciclo de vida de ML; además la deuda técnica es tangencial (CE4).
R16	Moreschini et al. (2026). The evolution of technical debt from DevOps to generative AI: A multivocal literature review	CE1	Revisión multivocal (61 fuentes). Se traslada a Trabajos relacionados.
R19	Shivashankar et al. (2026). QualiTagger: Automating software quality categorization in issue trackers	CE4	Clasifica atributos de calidad en issue trackers; la deuda técnica es motivación y GPT-4o solo una línea base.
R20	Shivashankar et al. (2025). Enhancing task prioritization in software development issues tracking system	CE4	Clasificación automática de prioridad de issues con transformers; no estudia deuda técnica.
R22	Storey (2026). From technical debt to cognitive and intent debt: Rethinking software health in the age of AI	CE3	Artículo de posición sin revisión por pares (manuscrito).

Etapa 3: lectura del texto completo

Los 16 registros restantes se leyeron completos. Dos se excluyeron: el ensayo de MacArthur porque analiza la deuda retórica en la escritura y solo usa el software como analogía (CE5), porque su objeto es la deuda algorítmica en frameworks de aprendizaje profundo escritos por humanos y los LLMs aparecen solo como una de varias fuentes de embeddings (CE4).

Registro	Estudio	Decisión	Criterio	Motivo
R09	Fontana et al. (2025). Binary and multi-class classification of self-admitted technical debt: How far can we go?	Pendiente		Elegible por título y resumen; texto completo pendiente de obtención. Se reserva para el Hito 2.
R13	Li et al. (2025). Large language model ChatGPT versus small deep learning models for self-admitted technical debt detection: Why not together?	Pendiente		Elegible por título y resumen; texto completo pendiente de obtención. Se reserva para el Hito 2.
R15	MacArthur (2025). Large language models and the problem of rhetorical debt	Excluido	CE5	Ensayo de humanidades digitales sobre 'deuda retórica' en la escritura; el software aparece solo como analogía.
R21	Simon et al. (2026). Automated detection of algorithm debt in deep learning frameworks: An empirical study	Excluido	CE4	El objeto es la deuda algorítmica en frameworks de DL escritos por humanos; los LLMs solo aportan una de varias fuentes de embeddings.

Etapa 4: evaluación de la calidad

Los 12 estudios elegibles con texto completo se evaluaron con el instrumento de la tabla 12. Diez superaron el umbral de 3,5 puntos. Se excluyeron dos: Tkachuk, con 3,0 puntos, porque su experimento usa pocos fragmentos de código, una puntuación relativa al máximo del grupo sin pruebas estadísticas y solo aporta evidencia parcial a una pregunta; y Bunch y Taeb, con 2,5 puntos, porque propone un marco de gobernanza conceptual sin datos propios, sin validación y sin discusión de limitaciones.

ID	Estudio	QA1	QA2	QA3	QA4	QA5	QA6	Puntaje	Decisión
S01	Agarwal et al. (2026)	1	1	1	1	1	1	6,0	Seleccionado
S03	Huang et al. (2026)	1	1	1	1	1	1	6,0	Seleccionado
S04	Li et al. (2026)	1	1	1	1	1	1	6,0	Seleccionado
S06	Sun et al. (2026)	1	1	1	1	1	1	6,0	Seleccionado
S09	OBrien et al. (2024)	1	1	1	1	1	1	6,0	Seleccionado
S10	Wang et al. (2024)	1	1	1	1	1	1	6,0	Seleccionado
S02	Brahmi et al. (2026)	1	1	1	1	0,5	1	5,5	Seleccionado
S08	Tornhill et al. (2025)	1	0,5	1	0,5	1	1	5,0	Seleccionado
S07	Bhutani et al. (2025)	1	1	1	0,5	0	1	4,5	Seleccionado
S05	Patel et al. (2026)	1	0,5	0	0	1	1	3,5	Seleccionado
R24	Tkachuk (2025)	1	0,5	0,5	0,5	0,5	0	3,0	Excluido
R08	Bunch y Taeb (2026)	1	0,5	0	0	0	1	2,5	Excluido

Tabla 12 - *Evaluación de la calidad de los estudios elegibles*

5.4. Filtros y aplicación de CI/CE

Los filtros se aplicaron en dos momentos. En la búsqueda, el único filtro fue el campo de búsqueda: título, resumen y palabras clave (TITLE-ABS-KEY en Scopus y TS en Web of Science). No se restringió por año ni por tipo de documento en las bases, porque el tema es muy reciente y un filtro automático podía dejar fuera estudios de 2026 que aún estaban en proceso de indexación. Los criterios de año, tipo de publicación e idioma se aplicaron después, registro por registro, durante el cribado. Así se puede justificar cada exclusión, en lugar de que la base la haga de forma invisible.

5.5. Estudios Primarios Seleccionados

El proceso concluyó con 10 estudios primarios, que reúnen lo mejor de ambas bases: cinco procedentes solo de Scopus, cuatro indexados en las dos bases y uno solo de WoS (Tabla 16).

A cada uno se le asignó un identificador de S01 a S10, ordenado del más reciente al más antiguo, que se usa en el resto del documento.

ID	Referencia	Año	Venue	Titulo	Base	QA
S01	Agarwal et al. [5]	2026	MSR '26 (ACM)	Estudio empírico (minería de repositorios)	Solo Scopus	6,0
S02	Brahmi et al. [16]	2026	MSR '26 (ACM)	Estudio empírico (minería de repositorios)	Solo Scopus	5,5
S03	Huang et al. [17]	2026	MSR '26 (ACM)	Estudio empírico (minería de repositorios)	Solo Scopus	6,0
S04	Li, Q. et al. [18]	2026	ACM TOSEM	Propuesta y evaluación de técnica	Scopus y WoS	6,0
S05	Patel et al. [6]	2026	IEEE Access	Marco conceptual	Scopus y WoS	3,5
S06	Sun et al. [19]	2026	Journal of Systems and Software	Evaluación experimental de LLMs	Scopus y WoS	6,0
S07	Bhutani et al. [20]	2025	Applied Computer Systems	Evaluación experimental de LLMs	Solo WoS	4,5
S08	Tornhill et al. [7]	2025	FSE Companion '25 (ACM)	Propuesta y evaluación de técnica	Solo Scopus	5,0
S09	OBrien et al. [21]	2024	ICSE '24 (IEEE/ACM)	Evaluación experimental de LLMs	Solo Scopus	6,0
S10	Wang et al. [22]	2024	IEEE TSE	Propuesta y evaluación de técnica	Scopus y WoS	6,0

Tabla 13 - *Estudios Primarios seleccionados*

6. Resultados

6.1. Extraer los datos de investigación

Los datos de los 10 estudios se extrajeron y las cifras que se reportan en este documento se verificaron contra el texto completo de cada artículo. La Tabla 17 resume el objetivo, el método y el hallazgo principal de cada estudio; el formulario completo, con país, LLM evaluado, instrumentos de medición, limitaciones y evidencia por pregunta, está en la hoja *Extraccion* de la matriz de trabajo.

ID	Objetivo	Método y datos	Hallazgo principal

S01	Estimar el efecto causal de adoptar agentes de codificación sobre la velocidad y la calidad del software.	Diferencias en diferencias escalonadas con controles emparejados; ventana de 6 meses. Datos: AIDev: 401 repos sin IA previa + 606 controles; 117 repos con IA previa + 73 controles.	Advertencias de análisis estático +18 % y complejidad cognitiva +39 %, persistentes; ganancias de velocidad solo cuando el agente es la primera herramienta de IA del proyecto; efectos de duplicación pequeños e inconsistentes.
S02	Caracterizar la prevalencia, calidad, remediación y tipos del SATD escrito por agentes de IA frente al humano.	Análisis cuantitativo (Mann-Whitney, delta de Cliff, Kruskal-Wallis) y análisis temático inductivo. Datos: AIDev-Pop: 856 repositorios; 820 SATD de agentes y 1 991 humanos; 525 SATD analizados.	SATD menos prevalente en agentes (≈2,6 % por commit vs. 4,25 % humano); calidad similar salvo especificidad técnica; taxonomía de 10 temas y 34 subtemas: requisitos 27,0 %, código 17,3 %, diseño 15,6 %, dependencias y compatibilidad 13,2 %; categorías nuevas de infraestructura y pipeline, cobertura del prompt e incertidumbre de diseño.
S03	Comparar la calidad del código y el sentimiento de los revisores entre PR de agentes y de humanos.	Métricas tradicionales, métrica nueva de redundancia semántica (Max Redundancy Score) y análisis de sentimiento; Mann-Whitney. Datos: AIDev: 3 858 PR en Python (Dataset A) y 617 PR de crewAI (Dataset B).	Métricas tradicionales con diferencias mínimas; redundancia media máxima 0,2867 en agentes vs. 0,1532 en humanos (≈1,87×); los revisores reaccionan de forma neutral o positiva ante el código de agentes: deuda técnica silenciosa.
S04	Identificar y clasificar SATD de múltiples fuentes y categorías.	Marco IMPACT: aumento de datos con ChatGPT y pipeline de dos modelos ajustados; comparación con 3 métodos SOTA y 2 modelos base; ablación. Datos: SATD en comentarios, pull requests, issues y commits; 8 categorías; conjunto entre proyectos.	F1 medio de 0,927 (comentarios), 0,870 (PR) y 0,908 (commits); 0,813 entre proyectos; el aprendizaje en contexto (Flan-T5-XXL) no supera 0,351; el SATD de código se confunde con el de diseño.
S05	Proponer un marco de niveles de comprensión y una taxonomía de vacíos de conocimiento en el desarrollo asistido por IA.	Construcción teórica; modelos de degradación del conocimiento y de acumulación de deuda declarados como hipótesis falsables. Datos: Sin datos empíricos (literatura previa).	Deuda de comprensión: la organización nunca adquiere el aprendizaje del código generado; cinco vacíos (procedencia, casos límite, integración, evolución, seguridad); cinco indicadores propuestos y no medidos.
S06	Caracterizar las características de calidad no funcional del código generado por LLMs según ISO/IEC 25010.	Multimétodo: revisión de literatura, talleres con practicantes y evaluación empírica de parches. Datos: Revisión de 109 artículos; 2 talleres con industria; SWE-bench Lite (300 instancias, 11 repositorios).	Parches de referencia con 30 recomendaciones de mantenibilidad vs. 423, 420 y 414 en los generados; la regla de importación cíclica concentra 360–365 problemas nuevos; 4–6 hallazgos de seguridad vs. 2; los practicantes advierten que el código generado puede acelerar la deuda técnica.

S07	Comparar la calidad del código traducido de Java a Python por dos LLMs.	Traducción automática y análisis estático con SonarQube (mantenibilidad, fiabilidad, seguridad, complejidad, deuda). Datos: 500 instancias aleatorias de 1 695 problemas (XLCoST).	Problemas de mantenibilidad: Java 3 432, ChatGPT 1 500, DeepSeek 832; deuda de mantenibilidad de 42, 13 y 11 días; esfuerzo de remediación de 6 h 45 m, 1 h 30 m y 1 h; complejidad cognitiva de ChatGPT (2 907) superior a la del original (2 684).
S08	Automatizar la remediación de deuda técnica con refactorizaciones de LLM validadas.	Herramienta ACE con validación sintáctica, verificación del code smell y de CodeHealth, y comparación semántica. Datos: Más de 100 000 problemas reales de CodeHealth en código abierto.	El mejor LLM sin validación produce refactorizaciones correctas solo el 37 % de las veces; con validación la precisión sube al 98 % y la exhaustividad queda en 52 %; la calidad cae en funciones de más de 70 LoC.
S09	Determinar si los comentarios TODO en el prompt hacen que Copilot reproduzca o pague la deuda descrita.	Generación controlada con tres estrategias de prompt y etiquetado manual iterativo. Datos: 36 381 TODO de 102 424 repositorios; muestra de 380; 1 140 generaciones.	Copilot genera código con síntomas de SATD tanto con el TODO en el prompt como sin él; puede pagar la deuda bajo ciertas condiciones; acuerdo entre etiquetadores excelente (Kappa 0,9408; 0,8177; 0,9444).
S10	Detectar just-in-time los commits en los que se omitió un comentario TODO (TDReminder).	Dos codificadores neuronales (cambio de código y mensaje de commit); comparación con líneas base; estudio con usuarios. Datos: Commits de 10 000 repositorios populares de GitHub.	Supera a Random Forest en AUC +28,6 % y coste-efectividad +83,2 % (Python), y +28,1 % y +79,3 % (Java); los usuarios confirman su utilidad.

Tabla 14 - *Síntesis de la extracción de datos*

6.2. Estadísticas de la revisión

La Figura 2 compara los 26 registros únicos con los 10 estudios seleccionados. Aunque la ventana de búsqueda empieza en 2021, no se recuperó ningún registro anterior a 2024, y 16 de los 26 registros (62 %) y 6 de los 10 estudios seleccionados son de 2026. Esta concentración confirma que se trata de un campo emergente, que surge con la adopción masiva de asistentes y agentes de código.

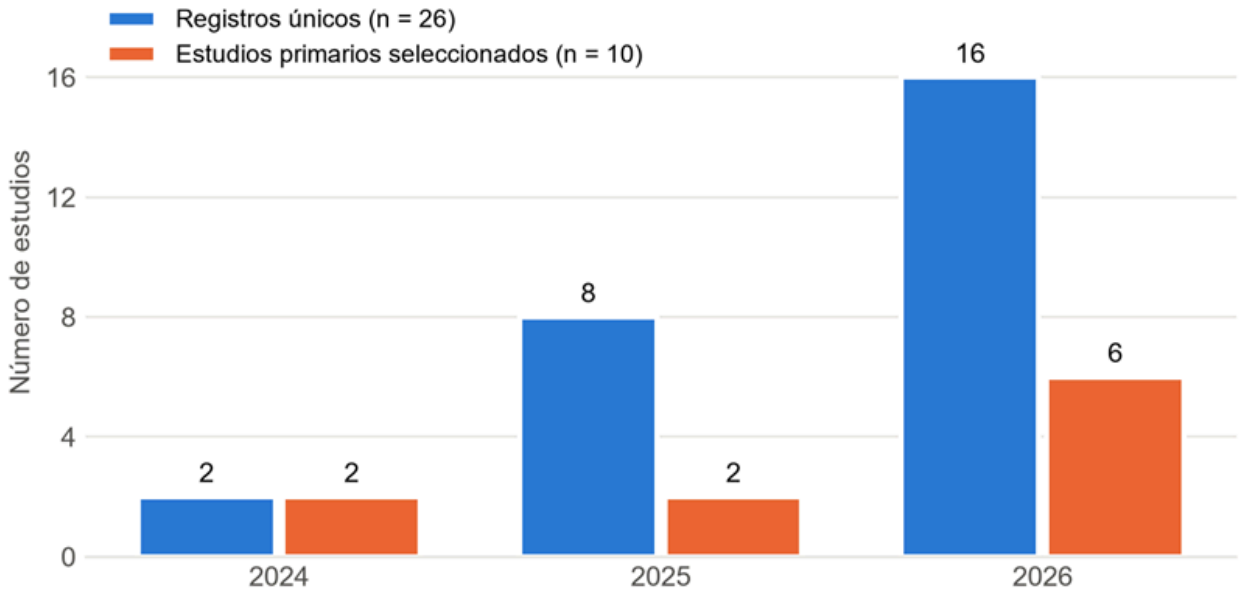


Figura 2 - Estudios por Año

La Figura 3 muestra el origen de los registros. WoS aportó más registros únicos (10 frente a 8 de Scopus), pero la mayoría fueron excluidos por ser estudios secundarios o por tratar la deuda técnica de forma tangencial. Entre los seleccionados predomina Scopus (9 de 10 estudios están indexados en Scopus), principalmente porque es la única base que recuperó las actas de conferencia.

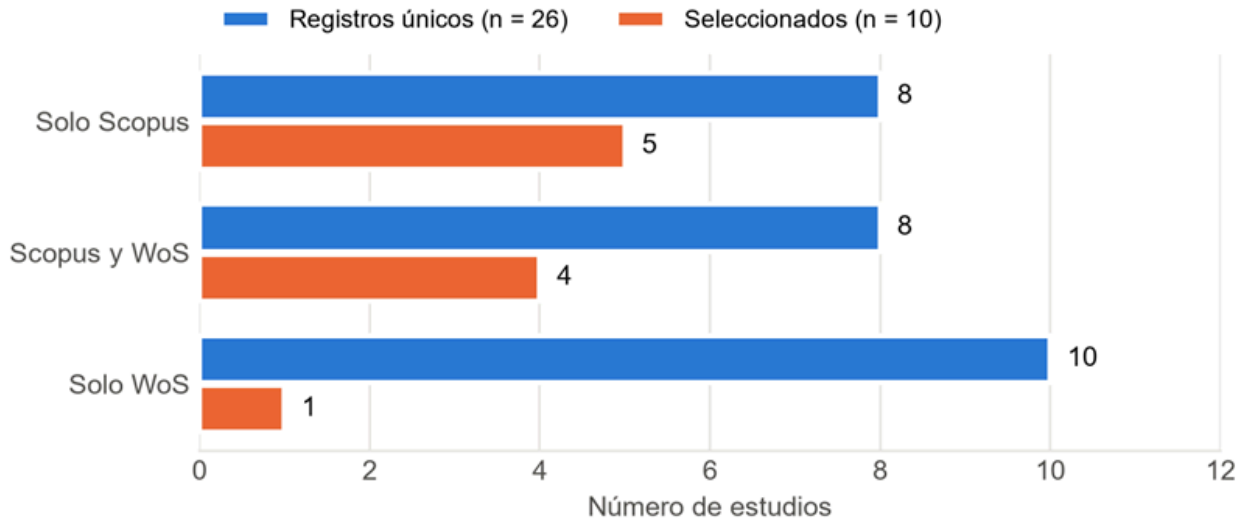


Figura 3 - Número de estudios por base de datos

La Figura 4 cuenta los estudios con al menos un autor afiliado a cada país. Estados Unidos participa en 4 estudios y Canadá en 3; les siguen China y Suecia con 2. Por afiliación del primer autor, Estados Unidos encabeza con 3 estudios, seguido de China y Suecia con 2, y de Canadá,

Japón e Irlanda con 1. Cuatro de los diez estudios son colaboraciones internacionales. No se identificaron estudios con autores de América Latina.

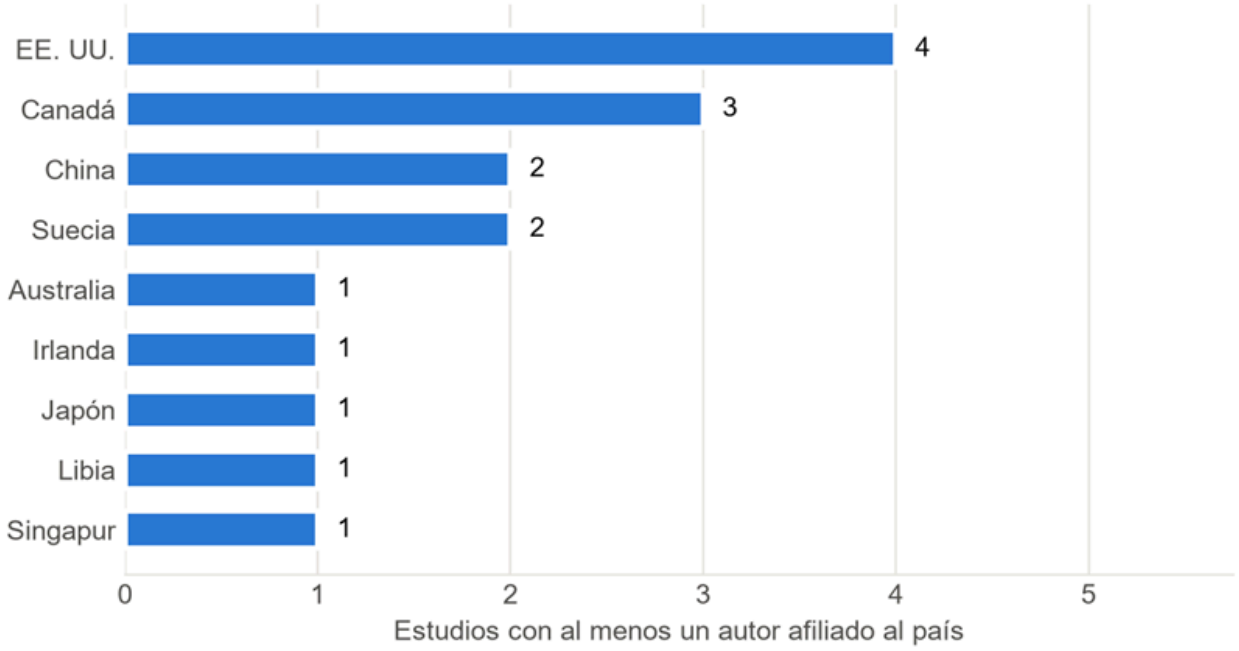


Figura 4 - Número de Estudio por país

La distribución es equilibrada, cinco artículos de conferencia y cinco de revista (Figura 5). Destaca la *International Conference on Mining Software Repositories* (MSR 2026), que concentra tres estudios, y la presencia de venues de alto impacto de la disciplina como ICSE, ACM TOSEM e IEEE TSE.

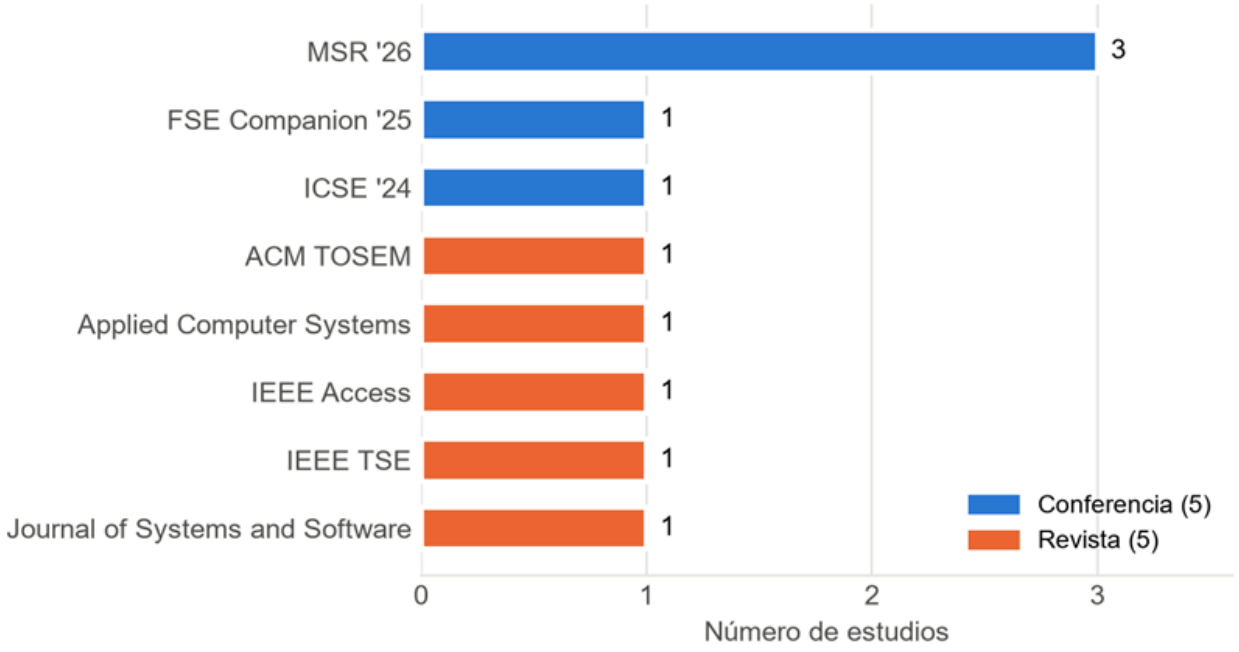


Figura 5 - Número de estudios por venue y tipo de publicación

Predomina la investigación empírica (Figura 6), tres estudios de minería de repositorios, tres evaluaciones experimentales de LLMs y tres propuestas de técnicas con evaluación; solo uno es un marco conceptual. Un dato relevante para la validez de la evidencia es que los tres estudios de minería de repositorios (S01, S02 y S03) usan el mismo conjunto de datos, AIDev, que recopila pull requests de agentes en GitHub.

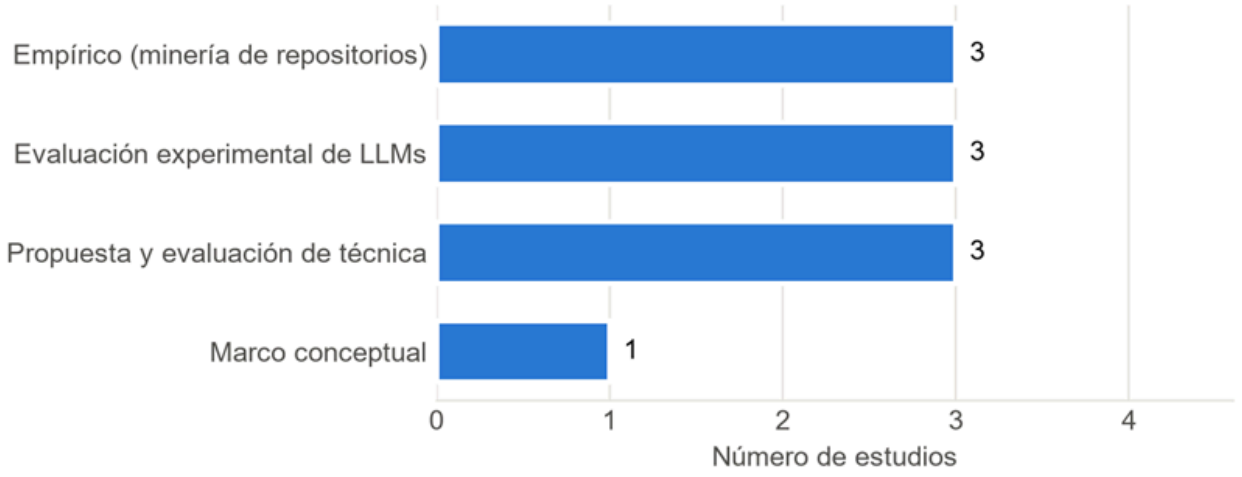


Figura 6 - Número de estudios por tipo de estudio

6.3. Caracterización de los estudios seleccionados

Antes de responder las preguntas conviene ver en qué se diferencian los estudios, porque no todos evalúan lo mismo: algunos analizan el código que produce un agente autónomo, otros el que sugiere un asistente en el IDE o un chat, y otros usan el LLM como herramienta para gestionar la deuda. La Tabla 15 compara los diez estudios según el tipo de asistencia, la herramienta o el modelo evaluado, el caso estudiado, su stack tecnológico y la evidencia que aportan a cada RQ; la Figura 11 muestra su distribución por tipo de asistencia.

ID	Tipo de asistencia	Herramienta o modelo evaluado	Caso estudiado	Stack tecnológico	RQ1 / RQ2 / RQ3
S01	Agente autónomo (vs. IDE)	Agentes autónomos (Codex, Claude Code, Devin, Cursor Agent) frente a AI IDEs	Repositorios open source de GitHub antes y después de adoptar agentes	AIDev · SonarQube · diferencias en diferencias	P / D / P
S02	Agente autónomo	Agentes Copilot, Cursor, Devin y Claude; Qwen2.5-7B como evaluador	Comentarios de deuda escritos por agentes en 856 repositorios	AIDev-Pop · comment-parser · Qwen2.5-7B-Instruct · pruebas no paramétricas	P / P / D
S03	Agente autónomo	Agentes de IA que generan pull requests (AIDev)	Pull requests de agentes frente a	AIDev · Radon · CodeSage-Large · PyRef · análisis de emociones	P / D / P

			humanos; caso crewAI		
S04	LLM como herramienta	ChatGPT (aumento de datos) y dos modelos de lenguaje ajustados (GLM-4-9B-Chat, MT-MoE-BERT)	SATD en comentarios, PR, issues y commits	ChatGPT (aumento de datos) · MT-MoE-BERT y GLM-4-9B-Chat ajustados	/ D / D
S05	Asistentes en general	Herramientas de generación de código (Copilot, CodeWhisperer, ChatGPT)	Análisis teórico		D / P / D
S06	Agente (SWE-agent)	GPT-4o, DeepSeek y Claude-Sonnet-4 (vía SWE-agent)	300 issues reales de SWE-bench Lite	SWE-agent · GPT-4o, DeepSeek, Claude-Sonnet-4 · Docker · CodeQL	P / D / P
S07	Chat / prompt directo	ChatGPT 4.1 y DeepSeek R1	Traducción de 500 programas de Java a Python	ChatGPT 4.1 · DeepSeek R1 · XLCoST · SonarQube	P / D / P
S08	LLM como herramienta	LLMs para refactorización (selección contextual de modelo)	Refactorización de funciones con code smells	CodeScene · CodeHealth · LLMs · validador de análisis estático	P / D /
S09	Asistente en el IDE	GitHub Copilot	380 comentarios TODO en funciones Python	GitHub Copilot · Boa (minería) · etiquetado manual	P / D / P
S10	LLM como herramienta	LLMs para generar recomendaciones explicables	Commits de 10 000 repositorios Python y Java	Codificadores neuronales · LLM para explicaciones · Python y Java	/ D / P

Tabla 15 - Resumen comparativo de los estudios primarios seleccionados

D = evidencia directa; P = evidencia parcial; = sin evidencia para esa pregunta.

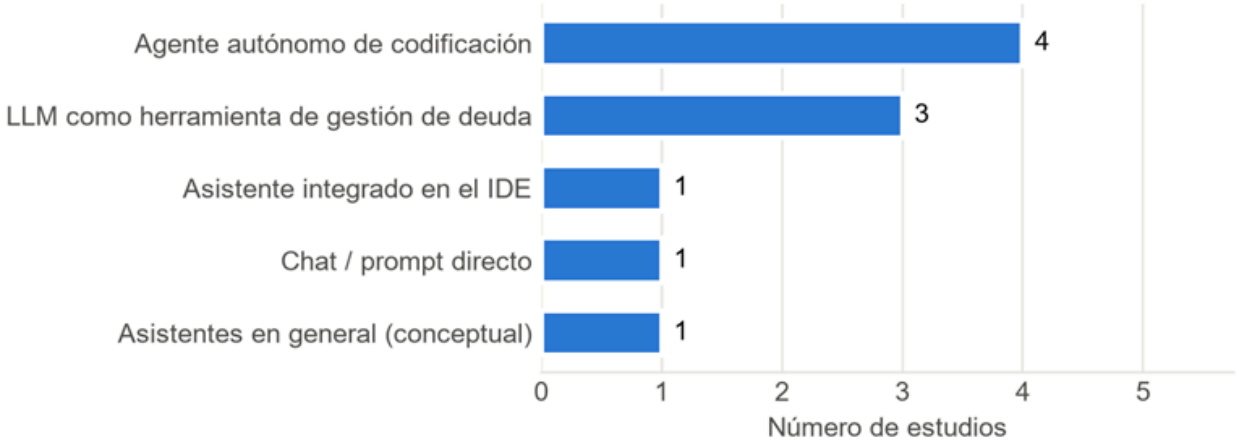


Figura 11 - Estudios por tipo de asistencia de IA evaluada

A continuación se describe cómo se realizó cada estudio:

S01 Agarwal et al. [5]. Estudio observacional longitudinal sobre repositorios de código abierto en GitHub. Tomaron del conjunto AIDev el primer pull request generado por un agente como fecha de adopción y separaron dos grupos: 401 repositorios sin uso previo de IA, comparados con 606 controles similares, y 117 repositorios que ya usaban asistentes en el IDE, comparados con 73 controles. Durante seis meses midieron cada mes la velocidad (commits, líneas añadidas) y la calidad con SonarQube, y estimaron el efecto con diferencias en diferencias. Es el único estudio que compara explícitamente agentes autónomos con asistentes en el IDE.

S02 Brahmi et al. [16]. Minería de 856 repositorios populares (AIDev-Pop) con contribuciones de los agentes Copilot, Cursor, Devin y Claude; Codex se excluyó porque su autoría no puede atribuirse con certeza. Con la librería comment-parser extrajeron los comentarios de los parches, identificaron la deuda autoadmitida con palabras clave y la depuraron a mano. Usaron el LLM Qwen2.5-7B-Instruct como evaluador de la especificidad y la calidad de cada comentario, compararon agentes y humanos con pruebas no paramétricas y construyeron una taxonomía mediante análisis temático.

S03 Huang et al. [17]. Analizó 3 858 pull requests en Python del conjunto AIDev y, como caso de estudio, 617 pull requests del repositorio crewAI. Calculó métricas tradicionales con Radon y una métrica nueva de redundancia: con embeddings del modelo CodeSage-Large midió la similitud entre cada función nueva y las ya existentes, filtrando con PyRef los movimientos y renombrados. Además analizó las emociones expresadas por los revisores en los comentarios de revisión.

S04 Li et al. [18]. Propone IMPACT, un marco para identificar y clasificar deuda autoadmitida en comentarios de código, pull requests, issues y mensajes de commit. Usa ChatGPT para generar datos de entrenamiento adicionales y encadena dos modelos de lenguaje ajustados (MT-MoE-BERT y GLM-4-9B-Chat), uno para identificar y otro para clasificar en ocho categorías. Lo compara con tres métodos del estado del arte y con los modelos base, incluido Flan-T5-XXL usado sin ajuste (aprendizaje en contexto), y lo prueba en proyectos no vistos.

S05 Patel et al. [6]. Estudio conceptual sin datos propios. A partir de la literatura, propone niveles de propiedad y comprensión del código, una taxonomía de cinco vacíos de conocimiento y modelos matemáticos de degradación del conocimiento, que los autores declaran como hipótesis por validar con estudios longitudinales.

S06 Sun et al. [19]. Combina tres métodos: una revisión de 109 artículos, dos talleres con profesionales de la industria y un experimento. En el experimento, el agente SWE-agent, con los modelos GPT-4o, DeepSeek y Claude-Sonnet-4, generó parches para los 300 issues de SWE-bench Lite (11 proyectos Python, principalmente Django y SymPy). Los parches se ejecutaron en contenedores Docker y se analizaron con CodeQL; también se midieron el tiempo de ejecución y la memoria, y se probaron prompts orientados a mejorar la calidad no funcional.

S07 Bhutani et al. [20]. Seleccionó al azar 500 de 1 695 problemas del conjunto XLCoST escritos en Java y los tradujo a Python con ChatGPT 4.1 y DeepSeek R1 mediante un prompt único, es decir, como un uso directo tipo chat. El código original y las dos traducciones se analizaron con SonarQube, que reporta problemas de mantenibilidad, fiabilidad y seguridad, complejidad y la deuda expresada en tiempo de remediación.

S08 Tornhill et al. [7]. Presenta ACE, una herramienta de CodeScene que se activa cuando un cambio empeora la salud del código. Un LLM genera varias refactorizaciones candidatas para cinco tipos de code smell y un validador análisis estático, verificación de que el smell desaparece y de que CodeHealth mejora, y comparación semántica descarta las incorrectas. Soporta JavaScript, TypeScript y Java, y se apoya en un estudio previo con más de 100 000 problemas reales y en pruebas con usuarios.

S09 OBrien et al. [21]. Extrajo con la infraestructura de minería Boa 36 381 comentarios TODO de 102 424 repositorios Python y tomó una muestra estadística de 380. Para cada uno pidió a GitHub Copilot, dentro del IDE, generar el cuerpo de la función con tres variantes de prompt que cambian la presencia y la forma del comentario TODO, obteniendo 1 140 generaciones. Dos autores etiquetaron a mano si el código reproducía o pagaba la deuda, con un tercero como moderador.

S10 Wang et al. [22]. Propone TDReminder, que detecta en el momento del commit si faltó añadir un comentario TODO. Dos codificadores neuronales representan el cambio de código y el mensaje de commit; el modelo se entrena con commits de 10 000 repositorios Python y 10 000 Java. Un LLM genera explicaciones sobre dónde y qué TODO añadir, y un estudio con usuarios evalúa su utilidad.

6.4. RQ1: ¿Cómo han cambiado el concepto, los mecanismos de acumulación y la gestión de la deuda técnica con la incorporación de los LLMs y la IA generativa al desarrollo de software?

Ocho de los diez estudios aportan evidencia para esta pregunta. Sus hallazgos se agrupan en siete cambios (Tabla 22 y Figura 12) y en ocho formas nuevas de gestión (Tabla 23). Para interpretarlos hay que tener presente el tipo de asistencia de cada estudio (Sección 6.3): la evidencia de más deuda proviene sobre todo de agentes autónomos (S01, S03, S06), mientras que el estudio de asistente en el IDE (S09) muestra un mecanismo distinto la reproducción de deuda aprendida y el de chat (S07) evalúa una tarea de traducción.

Categoría	Estudios que lo mencionan	N.º de estudios
Aumento persistente de complejidad y de problemas de mantenibilidad en el código generado	S01, S03, S06	3
Las ganancias de velocidad no compensan el riesgo de calidad	S01, S06, S08	3
El concepto clásico de deuda técnica (Cunningham; Dagstuhl) sigue vigente	S05, S08, S09	3
Reproducción de la deuda presente en los datos de entrenamiento o en el prompt	S02, S09	2
Menor visibilidad de la deuda: se admite o se detecta menos	S02, S03	2
La deuda surge en la generación y no de una decisión deliberada	S02, S05	2

Erosión de la comprensión del código dentro del equipo	S05	1

Tabla 16 - Cambios en la deuda técnica asociados al desarrollo asistido por LLMs

Lo que cambia no es qué es la deuda, sino cómo, a qué velocidad y con qué visibilidad se acumula.

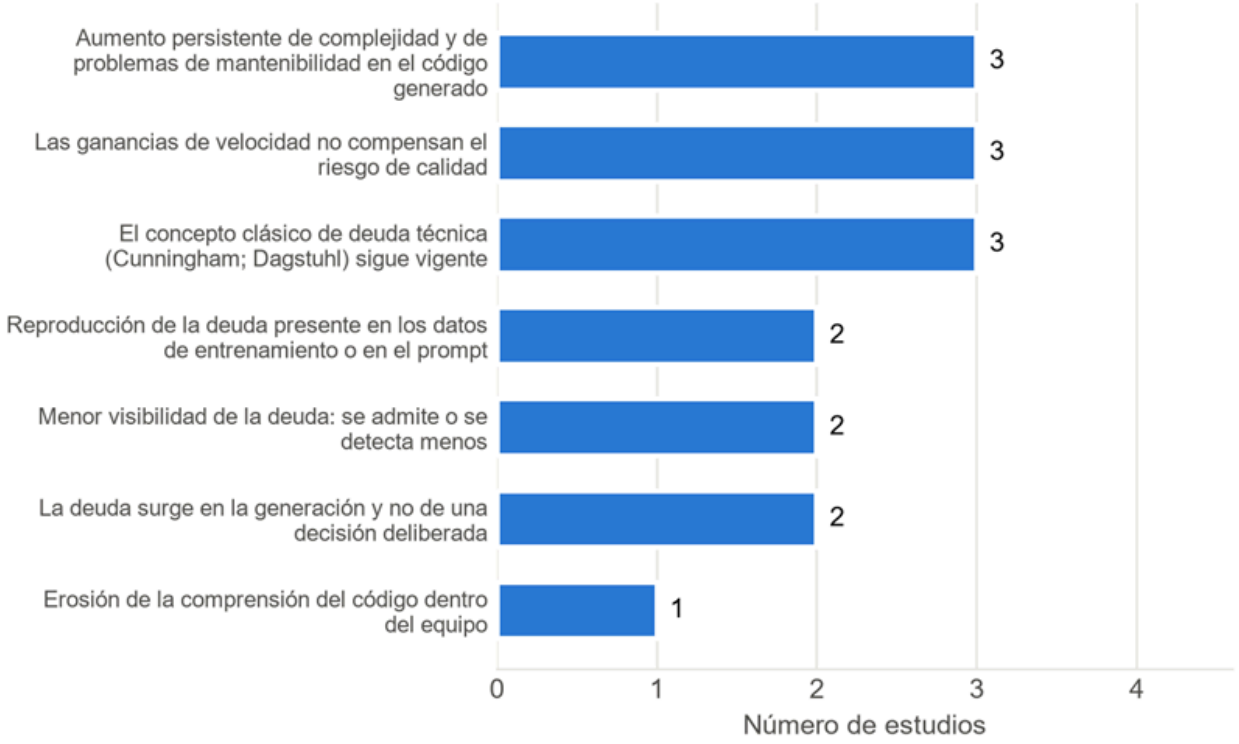


Figura 12 - *Número de estudios por cambio reportado (RQ1)*

Lo que cambia no es qué es la deuda, sino cómo, a qué velocidad y con qué visibilidad se acumula.

Más complejidad y más problemas de mantenibilidad

La evidencia más sólida es convergente y proviene de tres estudios independientes, todos sobre agentes (Figura 13). Agarwal et al. [5], con un diseño causal de diferencias en diferencias sobre 401 repositorios y 606 controles emparejados, encontraron que adoptar agentes de codificación eleva las advertencias de análisis estático en torno a 18 % y la complejidad cognitiva en torno a 39 %, y que ese aumento persiste durante los seis meses observados; los autores lo llaman deuda técnica inducida por agentes. Huang et al. [17] midieron que el código de los pull requests de agentes tiene una redundancia semántica 1,87 veces mayor que el humano (0,2867 frente a 0,1532). Sun et al. [19] observaron que los parches generados por GPT-4o, DeepSeek y Claude-Sonnet-4 disparan 423, 420 y 414 recomendaciones de mantenibilidad de CodeQL, frente a 30 en los parches escritos por los desarrolladores del proyecto.

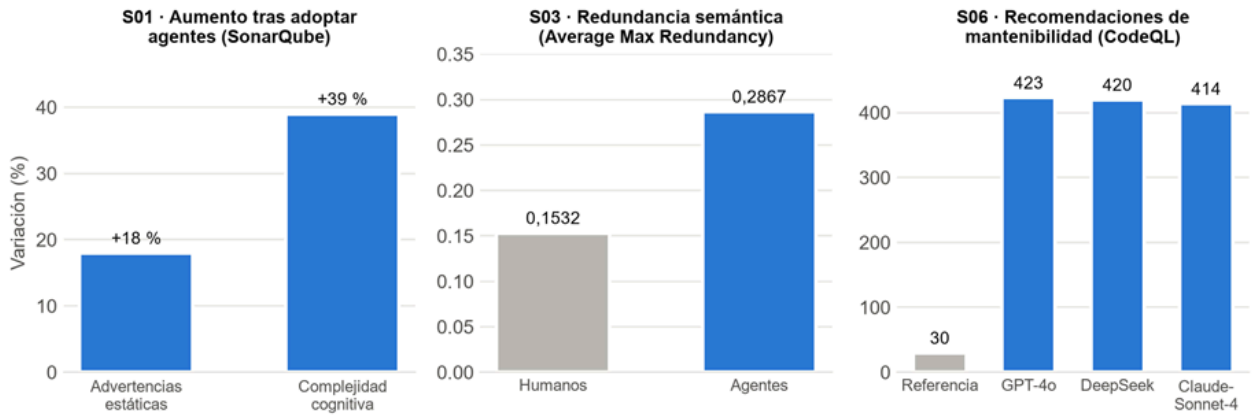


Figura 13 - Evidencia cuantitativa del aumento de deuda en código generado por agentes

Velocidad que no compensa la calidad

S01 muestra que las ganancias de velocidad son grandes solo cuando el agente es la primera herramienta de IA del proyecto; en repositorios que ya usaban asistentes en el IDE, las ganancias son mínimas o efímeras, mientras que los riesgos de calidad persisten en todos los escenarios. Es decir, pasar del IDE al agente aporta poca velocidad adicional pero sí más deuda. En los talleres de S06, los practicantes priorizan la mantenibilidad y advierten que el código generado puede acelerar la acumulación de deuda. S08 señala que los estudios que reportaron ganancias de productividad no midieron la calidad.

Deuda heredada del entrenamiento y del prompt

OBrien et al. [21] mostraron que GitHub Copilot, usado como asistente en el IDE, genera código con los síntomas de la deuda descrita en comentarios TODO, tanto cuando el comentario se incluye en el prompt como cuando no; los autores lo atribuyen a que el modelo reproduce la distribución de sus datos de entrenamiento, donde la deuda autoadmitida es frecuente. Brahmi et al. [16] confirman que los agentes replican los patrones de anotación de deuda aprendidos.

Menos visible y con otro origen

Dos hallazgos apuntan a un cambio en la visibilidad. Según S02, los agentes admiten deuda con menos frecuencia que los humanos (≈2,6 % de sus commits frente a 4,25 %), porque los humanos poseen el contexto, la responsabilidad y la intención sobre los compromisos de diseño; el SATD de los agentes no refleja decisiones deliberadas, sino que emerge del proceso de generación. Según S03, los revisores reaccionan de forma más neutral o positiva ante el código de agentes que ante el humano pese a su mayor redundancia: la plausibilidad superficial del código generado enmascara el problema y la deuda se acumula en silencio. Patel et al. [6] llevan el argumento más lejos, como hipótesis: con código generado como en el vibe coding la comprensión profunda puede no haber existido nunca en la organización. Mientras Cunningham describía una deuda de aprendizaje diferido, aquí el aprendizaje nunca se adquirió.

Categoría	Estudios que lo mencionan	N.º de estudios

Integrar análisis estático y aseguramiento de calidad en el pipeline de generación	S01, S06, S07	3
LLMs ajustados para identificar y clasificar SATD en varias fuentes	S04	1
Remediación automatizada con LLM y capa de validación	S08	1
Detección just-in-time en el commit con explicaciones generadas por LLM	S10	1
Usar el SATD dentro del prompt para pagar la deuda	S09	1
Métricas de redundancia semántica en la revisión de pull requests	S03	1
Trazabilidad de procedencia y despliegue selectivo de agentes	S01	1
Preservar el contexto de generación (prompt, alternativas, justificación)	S05	1

Tabla 17 - Nuevas formas de gestión de la deuda técnica en el contexto de los LLMs

Nuevas formas de gestión

La gestión se desplaza hacia la automatización con salvaguardas. S01, S06 y S07 coinciden en integrar el análisis estático y el aseguramiento de calidad dentro del flujo de generación (S01 propone exponer métricas de mantenibilidad en la planificación del propio agente). Las demás propuestas aparecen en un solo estudio cada una: clasificar SATD en varias fuentes con LLMs ajustados (S04); remediar deuda con refactorizaciones del LLM filtradas por una capa de validación (S08); detectar en el momento del commit que falta un TODO, con explicaciones generadas por un LLM (S10); usar el propio comentario de deuda en el prompt para que el asistente la pague (S09); medir la redundancia semántica en la revisión de pull requests (S03); rastrear la procedencia del código y desplegar agentes de forma selectiva (S01); y tratar el prompt y la justificación de la generación como artefactos bajo control de versiones (S05).

Síntesis de la RQ1. El concepto de deuda técnica se conserva, pero se amplía en tres dimensiones: la velocidad y el volumen con que se acumula (S01, S03, S06), su visibilidad, que disminuye porque se admite y se detecta menos (S02, S03), y su origen, que pasa de las decisiones deliberadas a la generación automática (S02, S05, S09). El efecto es más fuerte cuanto más autonomía tiene el LLM: los agentes concentran la evidencia de aumento de deuda. La gestión responde con automatización validada, aunque cada propuesta cuenta todavía con un solo estudio que la respalde.

6.5. RQ2: ¿Qué métricas y técnicas se emplean para medir y detectar la deuda técnica en el desarrollo asistido por LLMs, y qué evidencia de efectividad reportan?

Los diez estudios aportan evidencia para esta pregunta.

Categoría	Estudios que lo mencionan	N.º de estudios
Tamaño (LOC, líneas añadidas/eliminadas, tamaño de parche o de función)	S01, S03, S06, S07, S08	5
Complejidad (cognitiva o ciclomática)	S01, S03, S07, S08	4
Advertencias o reglas de análisis estático por severidad (code smells)	S01, S06, S07, S08	4
Duplicación y redundancia (sintáctica o semántica)	S01, S03	2
Esfuerzo de remediación (tiempo para pagar la deuda)	S02, S07	2
Seguridad (vulnerabilidades)	S06, S07	2
Calidad y prevalencia del SATD (FRE, GFI, especificidad, nivel de calidad)	S02	1
Salud agregada del código (CodeHealth)	S08	1
Fiabilidad (bugs)	S07	1
Eficiencia de rendimiento (tiempo de ejecución y memoria)	S06	1
Densidad de comentarios	S01	1
Indicadores de comprensión propuestos y no medidos	S05	1

Tabla 18 - Métricas empleadas para medir la deuda técnica

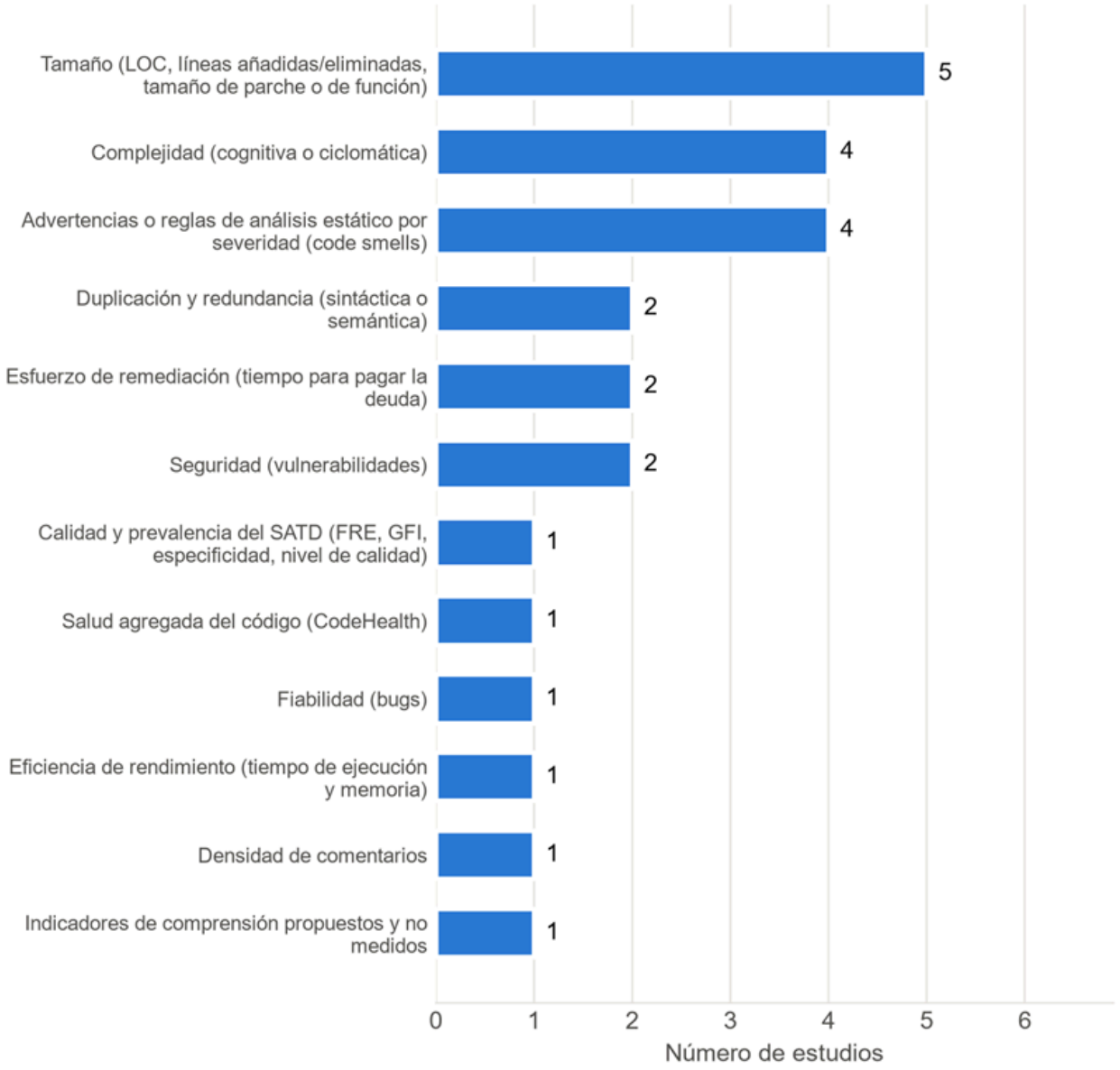


Figura 14 - Número de estudios por métrica empleada

Las métricas del SWEBOK siguen siendo la base, pero no todas son sensibles

Las métricas más usadas coinciden con los indicadores que el SWEBOK propone para estimar la deuda técnica: tamaño, complejidad y violaciones de buenas prácticas [1]. Sin embargo, su capacidad para distinguir la deuda que introducen los LLMs es desigual. Resultan sensibles la complejidad cognitiva y las advertencias de SonarQube (+39 % y +18 % en S01), las reglas específicas de mantenibilidad de CodeQL (la regla de importación cíclica concentra entre 360 y 365 problemas nuevos en S06) y la redundancia semántica (S03). Resultan poco sensibles la duplicación sintáctica, cuyos efectos en S01 son pequeños e inconsistentes, y la complejidad ciclomática y las métricas brutas de Radon, que en S03 no distinguen el código de agentes del humano; la única diferencia marcada está en lo que se elimina: los humanos borran más cadenas multilínea (26,26 frente a 8,47) y más líneas de código (25,51 frente a 16,60). La lección de S03 es que una métrica detecta la deuda de un LLM cuando está alineada con su

forma de fallar: los modelos probabilísticos tienden a producir clones semánticos (Tipo 4) que las herramientas basadas en coincidencia literal no ven.

Pocas mediciones expresan la deuda en tiempo o costo

Aunque el tópico del SWEBOK donde se ubica esta revisión es la estimación del costo de la deuda, solo dos estudios expresan la deuda como esfuerzo. Bhutani et al. [20] reportan con SonarQube una deuda de mantenibilidad de 42 días en el código Java original frente a 13 días en su traducción a Python con ChatGPT 4.1 y 11 días con DeepSeek R1. S02 mide el esfuerzo de remediación del SATD de agentes y encuentra diferencias significativas entre agentes (Kruskal-Wallis, H entre 38,8 y 82,7): los arreglos de Devin tardan en promedio 243,9 horas, más del triple que los de Cursor. Ningún estudio mide el acoplamiento sobre código generado, aunque S05 lo propone, junto con otros cuatro indicadores de comprensión, como dimensión para estudios futuros.

Categoría	Estudios que lo mencionan	N.º de estudios
Análisis estático como instrumento de medición (SonarQube, CodeQL, Radon)	S01, S03, S06, S07, S08	5
Palabras clave con validación manual para identificar SATD	S02, S09	2
Análisis manual o temático con calibración entre evaluadores	S02, S09	2
Pruebas estadísticas para comparar código de IA y humano	S02, S03	2
Modelos de lenguaje ajustados (fine-tuning) para clasificar SATD	S04	1
Modelos neuronales just-in-time sobre commits	S10	1
Validación programática de salidas del LLM	S08	1
Embeddings de código para detectar clones semánticos (Tipo 4)	S03	1
LLM como evaluador (LLM-as-a-judge)	S02	1
Inferencia causal (diferencias en diferencias)	S01	1

Tabla 18 - Técnicas de detección y evaluación de la deuda técnica

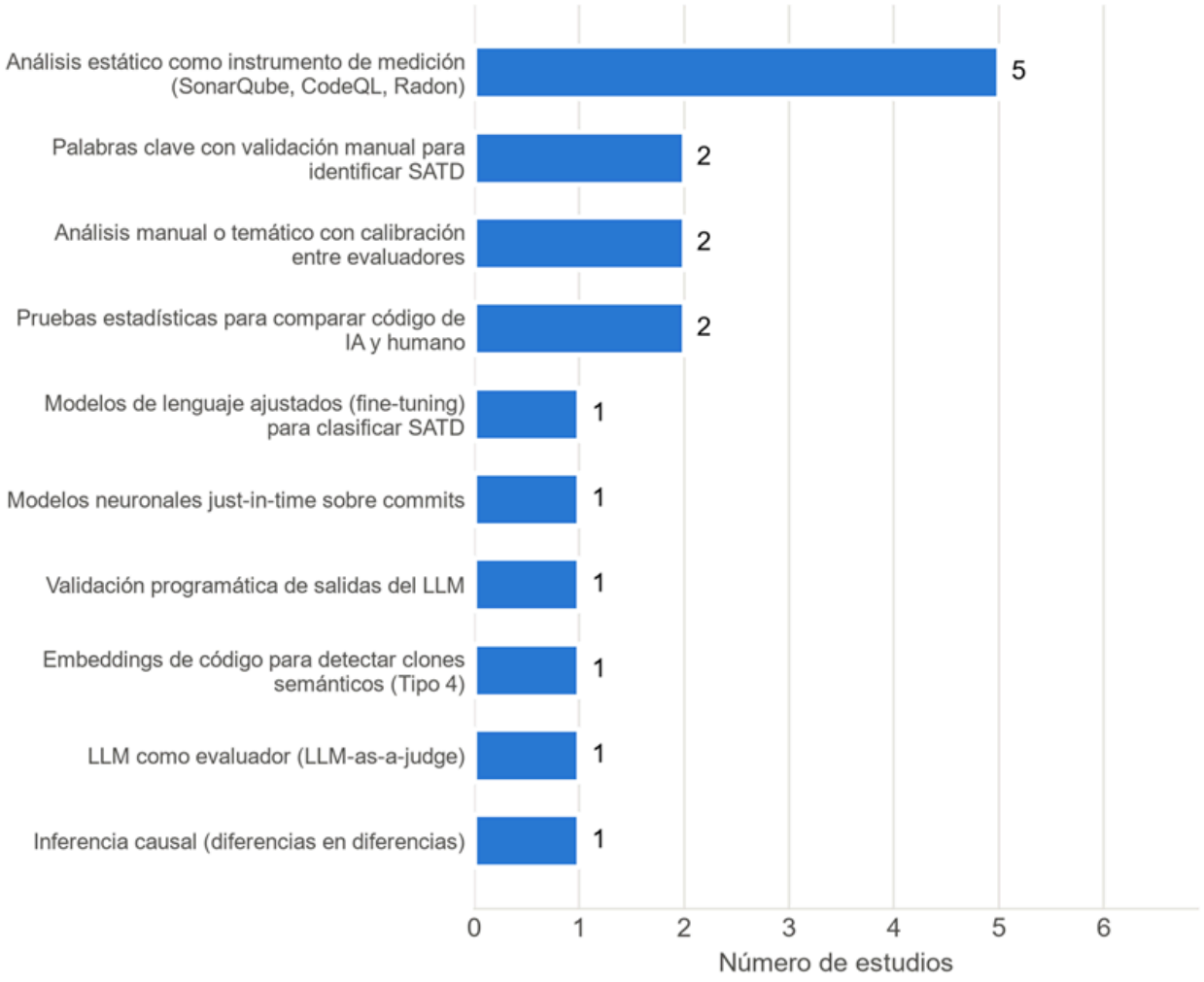


Figura 15 - Número de estudios por técnica de detección y evaluación

Técnicas de detección

El análisis estático (SonarQube, CodeQL, Radon y el validador de S08) es la técnica más usada, pero en cinco estudios funciona como instrumento de medición, no como detector evaluado: ningún estudio reporta su precisión frente a un criterio de referencia. Las técnicas que sí reportan efectividad son las basadas en aprendizaje. Li et al. [18] ofrecen la única comparación controlada: su marco IMPACT, que aumenta los datos con ChatGPT y ajusta dos modelos de lenguaje, alcanza un F1 de 0,927 en comentarios, 0,870 en pull requests y 0,908 en mensajes de commit, y 0,813 en proyectos no vistos, mientras que el mismo tipo de modelo usado con aprendizaje en contexto, sin ajuste, no supera 0,351. Wang et al. [22] detectan en el momento del commit que se omitió un TODO y superan a la mejor línea base en AUC (+28,6 % en Python y +28,1 % en Java).

Estudio	Técnica u objeto evaluado	Métrica	Resultado reportado
S04	Clasificación de SATD con LLMs ajustados (IMPACT)	Macro-F1	0,927 comentarios · 0,870 PR · 0,908 commits; 0,813 entre proyectos; aprendizaje en contexto ≤ 0,351

S08	LLM sin validación vs. con capa de validación (ACE)	Precisión / exhaustividad	37 % de refactorizaciones correctas sin validación; 98 % de precisión y 52 % de exhaustividad con validación
S10	Detección just-in-time (TDReminder)	Mejora relativa de AUC / coste-efectividad	+28,6 % / +83,2 % (Python); +28,1 % / +79,3 % (Java) frente a Random Forest
S01	Métricas de SonarQube ante la adopción de agentes	Efecto estimado	Advertencias +18 % y complejidad cognitiva +39 % (sensibles); duplicación sin efecto consistente
S03	Métricas tradicionales vs. redundancia semántica	Diferencia humano/agente	Métricas tradicionales con diferencias mínimas; redundancia 0,2867 vs. 0,1532 (≈1,87×)
S06	Reglas de mantenibilidad de CodeQL	Problemas introducidos	423 / 420 / 414 recomendaciones vs. 30 en parches de referencia; importación cíclica: 360–365
S07	Deuda de mantenibilidad de SonarQube	Días de remediación	Java original 42 días; ChatGPT 4.1 13 días; DeepSeek R1 11 días
S02	Identificación de SATD por palabras clave	Falsos positivos	103 de 820 (≈12 %), eliminados manualmente
S09	Etiquetado manual de generaciones	Kappa de Cohen	0,9408 · 0,8177 · 0,9444 (acuerdo excelente)

Tabla 19 - Efectividad reportada por las técnicas y métricas evaluadas

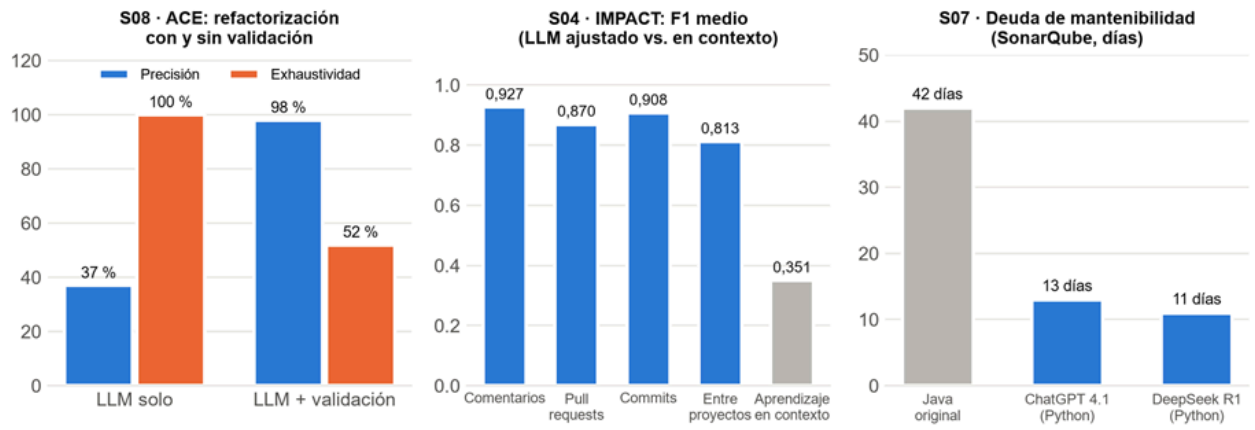


Figura 16 - Efectividad de las técnicas: validación en ACE (S08), F1 de IMPACT (S04) y deuda en días

La conclusión metodológica más robusta es que el LLM usado de forma aislada no es fiable para detectar o remediar deuda. En S08, el mejor LLM sin validación produce refactorizaciones funcionalmente correctas solo el 37 % de las veces; al filtrar sus salidas con una capa de validación (análisis estático, verificación del code smell y de CodeHealth, y comparación semántica) la precisión sube al 98 %, a costa de que la exhaustividad baje al 52 %; además, la calidad del LLM cae en funciones de más de 70 líneas. S04 apunta en la misma dirección: el aprendizaje en contexto es el enfoque más débil de los comparados. Las evaluaciones manuales siguen siendo necesarias, con procedimientos que reportan su fiabilidad: S09 etiquetó las generaciones de Copilot con un acuerdo excelente entre evaluadores (Kappa de 0,9408, 0,8177

y 0,9444), y S02 eliminó manualmente 103 falsos positivos (≈12 %) de la detección por palabras clave.

Síntesis de la RQ2. Los indicadores clásicos del SWEBOK siguen siendo la base de la medición, pero su sensibilidad a la deuda introducida por LLMs depende de que estén alineados con el mecanismo de generación: la complejidad cognitiva, las reglas específicas y la redundancia semántica la detectan; la duplicación sintáctica y la complejidad ciclomática, no. En detección, el ajuste fino y las capas de validación superan claramente al LLM sin supervisión. Persisten tres vacíos: no hay benchmarks comunes que permitan comparar técnicas entre estudios, no se mide el acoplamiento y casi no se estima la deuda en tiempo o costo.

6.6. RQ3: ¿Qué tipos de deuda técnica se reportan en el desarrollo asistido por LLMs y cuáles emergen específicamente de este contexto frente a la deuda técnica clásica?

Nueve estudios aportan evidencia para esta pregunta Los tipos reportados se separan en clásicos y emergentes.

Categoría	Estudios que lo mencionan	N.º de estudios
Deuda de código (code smells, baja calidad, complejidad)	S01, S02, S03, S04, S06, S07, S08, S09	8
Deuda de diseño y arquitectura	S02, S04, S06, S09	4
Deuda de documentación	S02, S04, S05, S09	4
Deuda de seguridad	S02, S05, S06, S07	4
Deuda de requisitos	S02, S04, S09	3
Deuda de pruebas	S02, S04, S09	3
Deuda de defectos	S02, S04, S09	3
Deuda de dependencias y compatibilidad	S02	1
Deuda de construcción (build)	S04	1
Deuda de rendimiento	S06	1

Tabla 20 - Tipos clásicos de deuda técnica reportados en el desarrollo asistido por LLMs

Los tipos clásicos se amplifican

La deuda de código aparece en 8 de los 10 estudios y es, con diferencia, la más reportada; le siguen la de diseño y arquitectura, la de documentación y la de seguridad, con 4 estudios cada una. La única distribución empírica del tipo de deuda que generan los agentes proviene de S02, que analizó 525 comentarios de SATD escritos por agentes (Figura 18): el tipo dominante es la deuda de requisitos (27,0 %), seguida de la de código (17,3 %) y la de diseño (15,6 %); las deudas de dependencias y de compatibilidad suman 13,2 %, y aparece una categoría nueva, infraestructura y pipeline (6,9 %). El predominio de la deuda de requisitos sugiere que los agentes tienen dificultades para inferir especificaciones implícitas o incompletas a partir del prompt. S04 trabaja con ocho categorías clásicas y muestra que la deuda de código y la de diseño están correlacionadas, lo que explica que los clasificadores las confundan. En seguridad,

S06 atribuye todos los problemas nuevos de los parches generados al almacenamiento de datos sensibles en texto plano.

Categoría	Estudios que lo mencionan	N.º de estudios
Deuda reproducida desde el prompt o los datos de entrenamiento	S02, S09	2
Deuda de complejidad inducida por agentes	S01	1
Deuda técnica silenciosa (redundancia semántica no detectada en la revisión)	S03	1
Deuda de comprensión y sus cinco vacíos de conocimiento	S05	1
SATD de agentes sobre infraestructura y pipelines (CI/CD, configuración, observabilidad)	S02	1
SATD de agentes por cobertura del prompt e incertidumbre de diseño	S02	1

Tabla 21 - Tipos de deuda que emergen específicamente del desarrollo asistido por LLMs

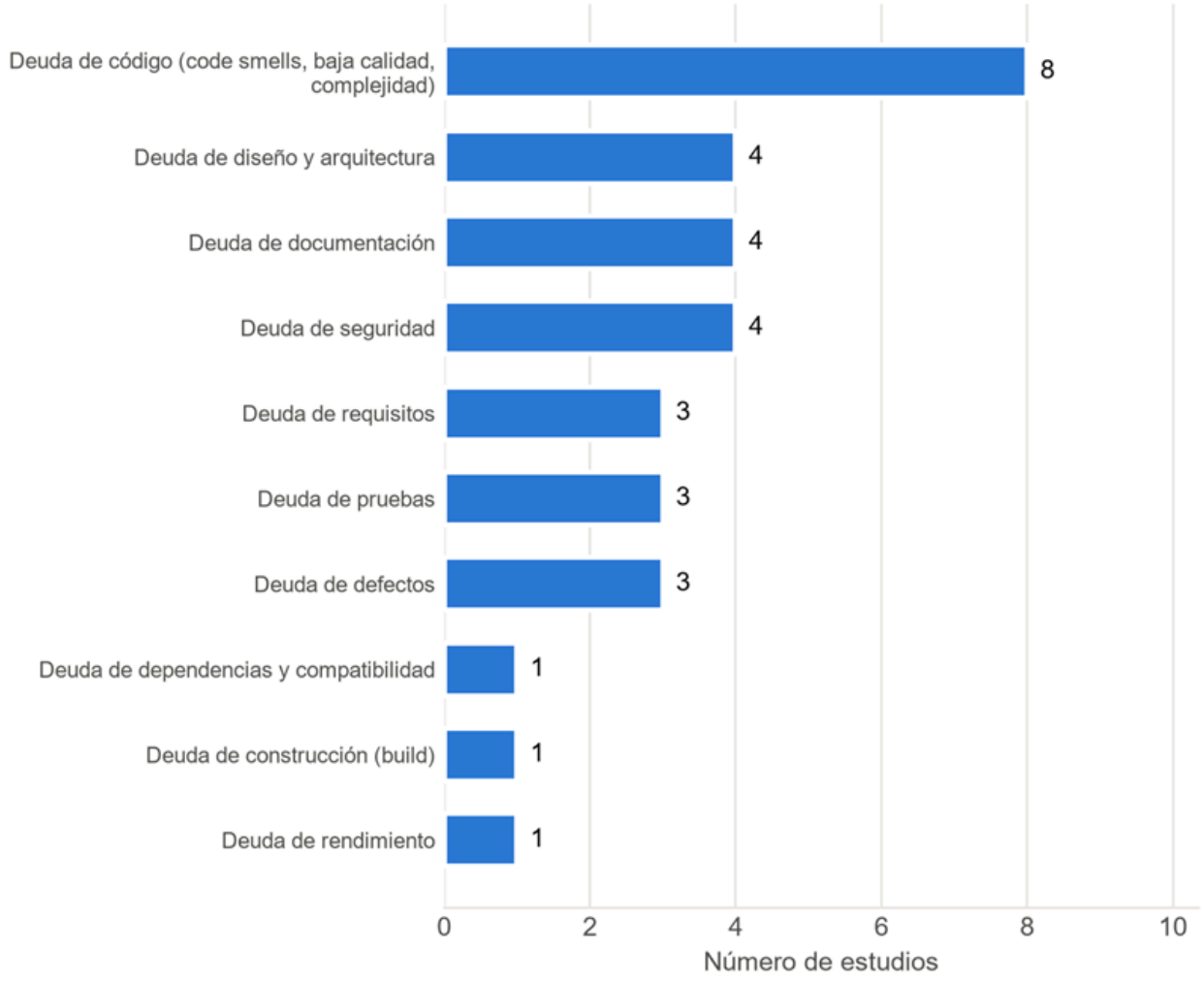


Figura 17 - Número de estudios por tipo clásico de deuda técnica

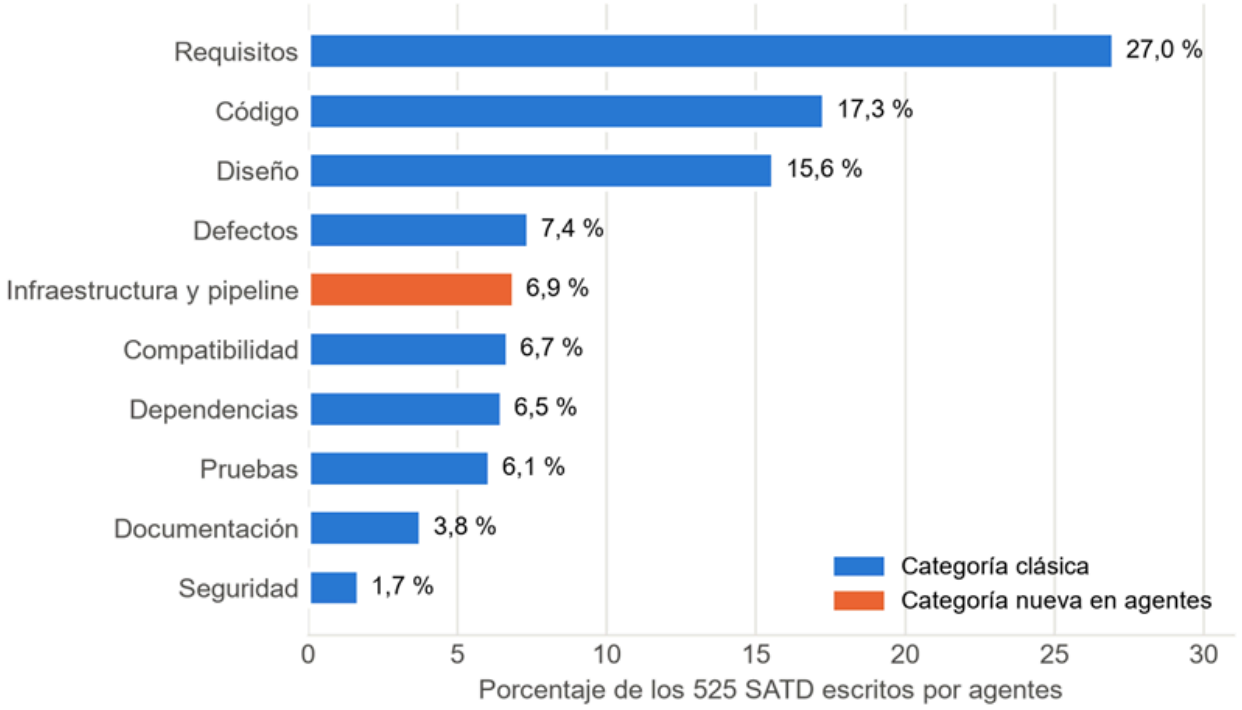


Figura 18 - Distribución de la deuda autoadmitida escrita por agentes, según Brahmi et al.

Tipos emergentes

Los tipos propios del contexto de los LLMs son pocos y cada uno descansa en uno o dos estudios:

- **Deuda reproducida desde el prompt o el entrenamiento** (S09, S02): el modelo reintroduce síntomas de deuda que aprendió de sus datos, con o sin incitación.
- **Deuda de complejidad inducida por agentes** (S01): el aumento persistente de complejidad cognitiva y advertencias tras adoptar agentes autónomos.
- **Deuda técnica silenciosa** (S03): redundancia semántica que no detectan ni las herramientas basadas en coincidencia literal ni los revisores humanos.
- **Deuda de comprensión** (S05): código en producción que el equipo no puede comprender ni modificar eficientemente, organizada en cinco vacíos de conocimiento procedencia, casos límite, integración, evolución y seguridad. Es una propuesta conceptual que sus autores declaran pendiente de validación.
- **SATD de agentes sobre infraestructura y pipelines** (S02): deuda sobre CI/CD, configuración, despliegue y observabilidad que los agentes identifican pero difieren por no conocer el entorno.
- **SATD de agentes por cobertura del prompt e incertidumbre de diseño** (S02): el agente declara dudas sobre el alcance de lo solicitado o sobre la decisión de diseño correcta. Son las categorías más genuinamente nuevas, porque no pueden existir sin un prompt.

Relación entre tipos. Como interpretación de esta revisión, los tipos emergentes no forman una familia separada, sino que se anclan en los clásicos: la cobertura del prompt es una forma de

deuda de requisitos; la incertidumbre de diseño, de deuda de diseño; la deuda inducida por agentes y la deuda silenciosa, de deuda de código; y la deuda de comprensión desplaza la deuda de documentación desde el código hacia las personas. Lo nuevo no es el tipo, sino el mecanismo que lo produce.

Síntesis de la RQ3. El desarrollo asistido por LLMs amplifica sobre todo los tipos clásicos, con la deuda de código presente en 8 de 10 estudios y la de requisitos como la más frecuente en el SATD de agentes. Los tipos emergentes existen, pero su respaldo es limitado: solo la taxonomía de S02 se derivó inductivamente de datos empíricos, y la deuda de comprensión sigue siendo una hipótesis.

6.7. Discusión

Convergencia de la evidencia empírica.

Tres estudios con datos y diseños independientes un diseño causal sobre repositorios de GitHub (S01), un análisis de pull requests de agentes (S03) y un experimento sobre SWE-bench Lite (S06) coinciden en que el código producido con LLMs acumula más complejidad y más problemas de mantenibilidad. Es la afirmación mejor respaldada del corpus.

Un resultado en tensión.

S07 parece contradecir lo anterior: el código traducido por LLMs acumuló menos deuda de mantenibilidad que el original (13 y 11 días frente a 42). La tensión se explica por el diseño: S07 compara reglas de SonarQube de lenguajes distintos (Java y Python) sobre salidas de distinto tamaño, y evalúa una tarea de traducción, no de generación. Incluso así, la complejidad cognitiva del código de ChatGPT (2 907) superó a la del original (2 684), en línea con S01, y el resultado depende del modelo: DeepSeek R1 produjo menos deuda que ChatGPT 4.1.

La paradoja de la visibilidad.

El código de agentes tiene más problemas (S01, S06), pero los agentes admiten menos deuda (S02) y los revisores la notan menos (S03). Esto tiene una consecuencia práctica: las técnicas de detección de SATD más efectivas (S04, S10) se entrenaron con comentarios escritos por humanos y aprenden de la deuda declarada. Si la deuda de los agentes es más implícita y además incluye categorías nuevas infraestructura, cobertura del prompt, esas técnicas podrían subestimarla. Ningún estudio del corpus lo ha evaluado.

Implicaciones para el SWEBOK.

Los indicadores que el SWEBOK propone para estimar la deuda (tamaño, complejidad y violaciones) siguen siendo los más usados y conservan validez, pero la evidencia sugiere complementarlos con métricas alineadas al modo de fallar de los LLMs, como la redundancia semántica. El SWEBOK recomienda además abordar la deuda desde sus causas raíz; los estudios identifican tres causas propias de este contexto: la autonomía de los agentes (S01), el punto ciego del revisor (S03) y los datos de entrenamiento (S09). La mayor brecha está justamente en que casi ningún estudio estima el costo de la deuda en tiempo o dinero.

Concentración de la evidencia.

Tres de los diez estudios comparten el conjunto de datos AIDev, la mayoría analiza proyectos de código abierto y Python es el lenguaje predominante. Los resultados podrían no trasladarse a entornos industriales ni a otros lenguajes, como advierten los propios autores de S03 y S06.

7. Conclusiones

Este estudio de mapeo sistemático, conducido según Kitchenham y Charters sobre Scopus y Web of Science, partió de 34 registros y seleccionó 10 estudios primarios publicados entre 2024 y 2026. Sus conclusiones, por pregunta de investigación, son las siguientes.

RQ1. El concepto de deuda técnica no cambia: los estudios mantienen la metáfora de Cunningham y la definición de Dagstuhl. Lo que cambia es su dinámica. El desarrollo asistido por LLMs acumula deuda más rápido (S01, S03, S06), la hace menos visible porque los agentes la admiten menos y los revisores la notan menos (S02, S03), y desplaza su origen desde las decisiones deliberadas hacia la generación automática (S02, S05, S09). El efecto es mayor cuanto más autónomo es el asistente. La gestión responde con automatización validada e integración de controles de calidad en el flujo de generación.

RQ2. Los indicadores clásicos del SWEBOK tamaño, complejidad y violaciones siguen siendo la base de la medición, pero solo algunos son sensibles a la deuda de los LLMs: la complejidad cognitiva, las reglas específicas de mantenibilidad y la redundancia semántica. En detección, el LLM sin supervisión no es fiable (37 % de refactorizaciones correctas en S08; F1 ≤ 0,351 con aprendizaje en contexto en S04), mientras que el ajuste fino y las capas de validación elevan la efectividad (F1 de hasta 0,927 en S04; 98 % de precisión en S08).

RQ3. Se amplifican sobre todo los tipos clásicos la deuda de código aparece en 8 de 10 estudios y la de requisitos domina el SATD de los agentes. Emergen categorías nuevas ligadas al mecanismo de generación: deuda inducida por agentes, deuda silenciosa, deuda de comprensión y SATD por cobertura del prompt, incertidumbre de diseño e infraestructura; su respaldo empírico, sin embargo, es aún limitado.

En conjunto, la revisión identifica una brecha directamente vinculada al área de conocimiento elegida: la estimación del costo de la deuda técnica en código generado por LLMs está prácticamente inexplorada, junto con la medición del acoplamiento, la ausencia de benchmarks comunes, la falta de comparaciones entre chat, IDE y agentes sobre la misma tarea y la falta de estudios longitudinales en la industria.

8. Referencias

[1] H. Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0a*. Los Alamitos, CA, USA: IEEE Computer Society, 2026. [Online]. Available: https://computer.org/swebok

[2] W. Cunningham, "The WyCash portfolio management system," in *Addendum to the Proc. Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, 1992, pp. 29–30, doi: 10.1145/157709.157715.

[3] P. Avgeriou, P. Kruchten, I. Ozkaya, and C. Seaman, "Managing technical debt in software engineering (Dagstuhl Seminar 16162)," *Dagstuhl Reports*, vol. 6, no. 4, pp. 110–138, 2016, doi: 10.4230/DagRep.6.4.110.

[4] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2014, pp. 91–100, doi: 10.1109/ICSME.2014.31.

[5] S. Agarwal, H. He, and B. Vasilescu, "AI IDEs or autonomous agents? Measuring the impact of coding agents on software development," in *Proc. 23rd Int. Conf. Mining Software Repositories (MSR '26)*, 2026, doi: 10.1145/3793302.3793589.

[6] M. S. Patel, R. Dhawan, L. Bramhanwade, and D. J. Mishra, "Code comprehension gaps in AI-assisted development: A conceptual framework for knowledge erosion patterns," *IEEE Access*, vol. 14, pp. 66740–66757, 2026, doi: 10.1109/ACCESS.2026.3689574.

[7] A. Tornhill, M. Borg, N. Hagatulah, and E. Söderberg, "ACE: Automated technical debt remediation with validated large language model refactorings," in *Companion Proc. 33rd ACM Int. Conf. Foundations of Software Engineering (FSE Companion '25)*, 2025, pp. 1318–1324, doi: 10.1145/3696630.3730565.

[8] K. Petersen, R. Feldt, S. Mujtaba, and M. Mattsson, "Systematic mapping studies in software engineering," in *Proc. 12th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, 2008, doi: 10.14236/ewic/EASE2008.8.

[9] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele Univ. and Durham Univ., Tech. Rep. EBSE-2007-01, 2007.

[10] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012, doi: 10.1109/MS.2012.167.

[11] O. Bunch and M. Taeb, "Trustworthiness of vibe coding from the NIST AI RMF perspective," in *Proc. ACM Southeast Conf. (ACMSE 2026)*, 2026, pp. 229–234, doi: 10.1145/3746467.3801530.

[12] S. Moreschini, E. Arvanitou, E.-P. Kanidou, N. Nikolaidis, R. Su, A. Ampatzoglou, A. Chatzigeorgiou, and V. Lenarduzzi, "The evolution of technical debt from DevOps to generative AI: A multivocal literature review," *J. Systems and Software*, vol. 231, Art. no. 112599, 2026, doi: 10.1016/j.jss.2025.112599.

[13] S. Gosu, D. Feitosa, and P. Avgeriou, "The use of generative AI in technical debt management: A systematic literature review," *Information and Software Technology*, vol. 199, Art. no. 108257, 2026, doi: 10.1016/j.infsof.2026.108257.

[14] A. Abdulaziz, R. Sonbol, and M. Alnoukari, "Automated detection of self-admitted technical debt: A systematic literature review," *IEEE Access*, vol. 14, pp. 20803–20825, 2026, doi: 10.1109/ACCESS.2026.3661069.

[15] I. Kosmas, T. Papadopoulos, and C. Michalakelis, "Machine learning lifecycle: A survey," *AppliedMath*, vol. 6, no. 7, Art. no. 113, 2026, doi: 10.3390/appliedmath6070113.

[16] Z. Brahmi, A. Ouni, M. Sayagh, and M. A. Saied, "Characterizing self-admitted technical debt generated by AI coding agents," in *Proc. 23rd Int. Conf. Mining Software Repositories (MSR '26)*, 2026, pp. 852–856, doi: 10.1145/3793302.3793588.

[17] H. Huang, P. Jaisri, S. Shimizu, L. Chen, S. Nakashima, and G. Rodríguez-Pérez, "More code, less reuse: Investigation on code quality and reviewer sentiment towards AI-generated pull requests," in *Proc. 23rd Int. Conf. Mining Software Repositories (MSR '26)*, 2026, pp. 1024–1028, doi: 10.1145/3793302.3793622.

[18] Q. Li, Z. Yin, Y. Yang, C. Li, Z. Shen, J. Ge, W. Zhong, B. Luo, and V. Ng, "IMPACT: Identifying and classifying multiple sourced and categorized self-admitted technical debts," *ACM Trans. Software Engineering and Methodology*, vol. 35, no. 4, Art. no. 102, 2026, doi: 10.1145/3747180.

[19] X. Sun, D. Ståhl, K. Sandahl, and C. Kessler, "Quality assurance of LLM-generated code: Addressing non-functional quality characteristics," *J. Systems and Software*, vol. 238, Art. no. 112885, 2026, doi: 10.1016/j.jss.2026.112885.

[20] V. Bhutani, F. G. Toosi, and J. Buckley, "Analysing software quality of AI-translated code: A comparative study of large language models using static analysis," *Applied Computer Systems*, vol. 30, no. 1, pp. 105–121, 2025, doi: 10.2478/acss-2025-0013.

[21] D. OBrien, S. Biswas, S. M. Imtiaz, R. Abdalkareem, E. Shihab, and H. Rajan, "Are prompt engineering and TODO comments friends or foes? An evaluation on GitHub Copilot," in *Proc. 46th IEEE/ACM Int. Conf. Software Engineering (ICSE '24)*, 2024, doi: 10.1145/3597503.3639176.

[22] H. Wang, Z. Gao, X. Hu, D. Lo, J. Grundy, and X. Wang, "Just-in-time TODO-missed commits detection," *IEEE Trans. Software Engineering*, vol. 50, no. 11, pp. 2732–2752, 2024, doi: 10.1109/TSE.2024.3405005.

9. Anexos

Anexo A. Inventario de estudios primarios seleccionados

Los estudios se ordenan del más reciente al más antiguo. Los archivos PDF se entregan con el nombre de su identificador (S01_…pdf, S02_…pdf, etc.).

ID	Título	Primer autor	Año
S01	AI IDEs or autonomous agents? Measuring the impact of coding agents on software development	Agarwal	2026
S02	Characterizing self-admitted technical debt generated by AI coding agents	Brahmi	2026
S03	More code, less reuse: Investigation on code quality and reviewer sentiment towards AI-generated pull requests	Huang	2026
S04	IMPACT: Identifying and classifying multiple sourced and categorized self-admitted technical debts	Li	2026
S05	Code comprehension gaps in AI-assisted development: A conceptual framework for knowledge erosion patterns	Patel	2026
S06	Quality assurance of LLM-generated code: Addressing non-functional quality characteristics	Sun	2026

S07	Analysing software quality of AI-translated code: A comparative study of large language models using static analysis	Bhutani	2025
S08	ACE: Automated technical debt remediation with validated large language model refactorings	Tornhill	2025
S09	Are prompt engineering and TODO comments friends or foes? An evaluation on GitHub Copilot	OBrien	2024
S10	Just-in-time TODO-missed commits detection	Wang	2024

Tabla 22 - *Inventario de estudios primarios*

Enlace a la carpeta de estudios primarios:

Anexo B. Registro de decisiones de selección

Resumen de las decisiones tomadas sobre los 26 registros únicos. El detalle, con título, DOI y motivo de cada decisión, está en la hoja Seleccion_26 de la matriz de trabajo.

Registro	Estudio	Origen	Etapa 2	Etapa 3	Calidad	Decisión final
R01	Abdulaziz et al. (2026)	Scopus y WoS	Excluido (CE1)			Trabajo relacionado
R02	Agarwal et al. (2026)	Solo Scopus	Incluido	Incluido	6,0	Seleccionado (S01)
R03	Bagane et al. (2025)	Solo WoS	Excluido (CE4)			Excluido
R04	Beaucoral (2026)	Solo WoS	Excluido (CE5)			Excluido
R05	Bhatnagar (2026)	Solo WoS	Excluido (CE4)			Excluido
R06	Bhutani et al. (2025)	Solo WoS	Incluido	Incluido	4,5	Seleccionado (S07)
R07	Brahmi et al. (2026)	Solo Scopus	Incluido	Incluido	5,5	Seleccionado (S02)
R08	Bunch y Taeb (2026)	Solo Scopus	Incluido	Incluido	2,5	Excluido por calidad
R09	Fontana et al. (2025)	Solo WoS	Excluido (CE4)			Excluido
R10	Gosu et al. (2026)	Solo WoS	Excluido (CE1)			Trabajo relacionado
R11	Huang et al. (2026)	Solo Scopus	Incluido	Incluido	6,0	Seleccionado (S03)
R12	Kosmas et al. (2026)	Solo WoS	Excluido (CE1)			Trabajo relacionado
R13	Li et al. (2025)	Solo WoS	Incluido	Incluido	6,0	Seleccionado (S06)
R14	Li et al. (2026)	Scopus y WoS	Incluido	Incluido	6,0	Seleccionado (S04)

R15	MacArthur (2025)	Scopus y WoS	Incluido	Excluido (CE5)		Excluido
R16	Moreschini et al. (2026)	Scopus y WoS	Excluido (CE1)			Trabajo relacionado
R17	OBrien et al. (2024)	Solo Scopus	Incluido	Incluido	6,0	Seleccionado (S09)
R18	Patel et al. (2026)	Scopus y WoS	Incluido	Incluido	3,5	Seleccionado (S05)
R19	Shivashankar et al. (2026)	Solo WoS	Excluido (CE4)			Excluido
R20	Shivashankar et al. (2025)	Solo WoS	Excluido (CE4)			Excluido
R21	Simon et al. (2026)	Scopus y WoS	Incluido	Excluido (CE4)		Excluido
R22	Storey (2026)	Solo Scopus	Excluido (CE3)			Excluido
R23	Sun et al. (2026)	Scopus y WoS	Incluido	Incluido	6,0	Seleccionado (S06)
R24	Tkachuk (2025)	Solo Scopus	Incluido	Incluido	3,0	Excluido por calidad
R25	Tornhill et al. (2025)	Solo Scopus	Incluido	Incluido	5,0	Seleccionado (S08)
R26	Wang et al. (2024)	Scopus y WoS	Incluido	Incluido	6,0	Seleccionado (S10)

Tabla 23 - *Registro de decisiones sobre los 26 registros únicos*

Anexo C. Justificación de la evaluación de calidad

Estudio	Puntaje	Justificación
S01 · Agarwal et al. (2026)	6,0	Objetivo causal explícito; diferencias en diferencias escalonadas con controles emparejados; 401+606 y 117+73 repositorios (AIDev); errores estándar y significancia; discute tendencias previas y límites del diseño; evidencia directa en RQ2.
S02 · Brahmi et al. (2026)	5,5	Tres RQs; análisis cuantitativo y temático inductivo; 856 repositorios, 820 SATD de agentes y 1 991 humanos; Mann-Whitney, delta de Cliff y Kruskal-Wallis; sin sección de amenazas (solo reconoce exclusión de Codex y falsos positivos); directa en RQ3.
S03 · Huang et al. (2026)	6,0	Dos RQs; métricas Radon, métrica de redundancia semántica y análisis de sentimiento; 3 858 y 617 PR; Mann-Whitney y verificación manual; amenazas internas y externas; directa en RQ2.
S04 · Li et al. (2026)	6,0	Objetivos y RQs claros; comparación controlada con 3 métodos SOTA y 2 modelos base; datos multifuente y conjunto entre proyectos; macro-F1 y ablación; sección de amenazas; directa en RQ2 y RQ3.
S09 · OBrien et al. (2024)	6,0	RQs explícitas; generación controlada con tres estrategias de prompt; 36 381 TODO, muestra de 380 (95 %, ±5 %), 1 140

		generaciones; Kappa 0,94/0,82/0,94; amenazas a la validez; directa en RQ2.
S06 · Sun et al. (2026)	6,0	Multimétodo (revisión de 109 artículos, 2 talleres, experimento); SWE-bench Lite (300 instancias) con 3 LLMs; CodeQL por severidad frente a parches de referencia; amenazas (§6); directa en RQ2.
S10 · Wang et al. (2024)	6,0	Tarea y objetivo definidos; modelo neuronal con líneas base; 10 000 repositorios Python y Java; AUC, coste-efectividad y estudio con usuarios; amenazas (§7.1); directa en RQ2.
S08 · Tornhill et al. (2025)	5,0	Objetivo claro; artículo de herramienta cuya evaluación se apoya en un benchmark previo (>100 000 problemas) y en retroalimentación temprana; reporta precisión y exhaustividad sin análisis estadístico propio; sección de limitaciones; directa en RQ2.
S07 · Bhutani et al. (2025)	4,5	Objetivo claro; muestreo aleatorio de 500 de 1 695 problemas y análisis con SonarQube; resultados descriptivos agregados sin pruebas estadísticas; no discute amenazas a la validez; directa en RQ2.
S05 · Patel et al. (2026)	3,5	RQs explícitas; construcción conceptual con modelos declarados como hipótesis; sin datos ni resultados empíricos; limitaciones explícitas; directa en RQ1 y RQ3.
R24 · Tkachuk (2025)	3,0	Objetivo claro; cinco tareas con puntuación normalizada relativa al máximo colectivo; fragmentos Swift y un archivo Java; sin pruebas estadísticas; menciona brevemente la no reproducibilidad; solo evidencia parcial en RQ2.
R08 · Bunch y Taeb (2026)	2,5	Objetivo claro; síntesis de literatura no sistemática y marco de gobernanza conceptual; sin datos propios ni validación; no discute limitaciones; directa en RQ1 y RQ3.

Tabla 24 - *Justificación del puntaje de calidad por estudio*